

Received 10 May 2026, accepted 22 May 2026, date of publication 26 May 2026, date of current version 2 June 2026.

Digital Object Identifier 10.1109/ACCESS.2026.3697295

RESEARCH ARTICLE

Chess Board Digitization and Human Move Prediction Using Machine Learning

HEIGON SOLDERA^{ID} AND HELIO PEDRINI^{ID}, (Senior Member, IEEE)

Institute of Computing, University of Campinas, Campinas 13083-852, Brazil

Corresponding author: Helio Pedrini (helio@ic.unicamp.br)

ABSTRACT Chess is a strategic game widely studied due to its complexity and the cognitive skills it requires. The analysis of chess games has been significantly enhanced by the use of machine learning techniques, allowing not only the creation of digital boards that identify the positions of the pieces in real-time but also the determination of the best possible moves. These tools enable the development of advanced training systems and the offering of complex strategies, making the understanding of moves more challenging. This project proposes the integration of two main fronts: the digitization of chess boards through images and the prediction of moves that mimic human patterns. The approach combines computer vision techniques, machine learning, and realistic three-dimensional (3-D) models, allowing the capture of arbitrary configurations of the pieces on the board and suggesting moves that simulate the style of professional players. The proposed system introduces a human-move prediction model that acts as a pre-filter, guiding the move selection process before validation by a traditional engine. The goal is to offer an accessible solution for board digitization and propose more understandable moves. In digitization, the methodology employed uses photogrammetry of chess pieces and the board to generate a large set of synthetic images from 3-D models that are used for pre-training the model. Refinement is carried out with a limited set of real images, allowing efficient learning with less dependence on real data. This approach resulted in an accuracy of 99.78% in the classification of pieces in the test set, reducing the effort of collecting real data and maintaining high quality in the results. For move prediction, a model based on convolutional neural networks was developed, capable of classifying the movements of the pieces based on patterns learned from human players' games. Rather than functioning as an independent chess engine, this model is integrated with Stockfish, which is responsible for evaluating and validating the final move selection. This model acts as a pre-filter for computational tools, which then select the best move. Under the proposed evaluation protocol, the hybrid approach achieved an average Top-1 accuracy of 68.52% in predicting human moves, compared to 39.38% obtained by Stockfish alone, representing an improvement of approximately 29 percentage points. This performance remains consistent across different game phases and shows competitive results when compared to prior human-move prediction approaches such as Maia, which reports an average accuracy of around 50% in a broader setting across multiple player rating groups and playing styles. Overall, the results demonstrate that combining learned human-like priors with engine-based validation improves both the plausibility and the practical quality of suggested moves.

INDEX TERMS Chess game, machine learning, neural networks, board digitization, human move prediction.

The associate editor coordinating the review of this manuscript and approving it for publication was Yiming Tang^{ID}.

I. INTRODUCTION

The game of chess has long held a fascination for mathematicians and computer scientists, serving as a benchmark for artificial intelligence capabilities since the earliest days

of computing. From Charles Babbage's Analytical Machine, where Ada Lovelace speculated about its potential to develop complex tasks such as composing music or playing chess [1], the challenge of chess has driven computational innovation. This rich history establishes chess as more than merely a game, it represents a domain where human cognition meets machine computation, offering insights into both strategic reasoning and artificial intelligence methodologies.

In the early 1950s, the first formal academic paper on computer chess emerged with Claude Shannon's seminal work, *Programming a Computer for Playing Chess* [2]. Shannon proposed several potential approaches to solving chess, including an algorithm utilizing a defined depth tree search where leaf nodes contained an evaluation function composed of a linear combination of factors affecting position quality. This foundation evolved into the Minimax algorithm with alpha-beta pruning [3], now widely disseminated across various engines, the computational tools designed to play chess. During this same period, Alan Turing and David Champernowne developed *Turochamp*, recognized as the first chess engine despite being manually simulated rather than computer-executed [4].

The technological evolution of chess engines represents a microcosm of artificial intelligence development more broadly. Early computer implementations emerged around 1970, as documented in *Kasparov versus Deep Blue: Computer Chess Comes of Age* [5]. While programming a computer to play chess presented moderate technical challenges, early game quality remained at amateur levels. Many Grandmasters (the highest title awarded by the International Chess Federation - FIDE) believed machines could never surpass this caliber. This perception shifted when Northwestern University's *CHESS 4.6* [6] defeated intermediate and advanced players in 1977, culminating in IBM's *Deep Blue* [7], a specialized multiprocessor machine analyzing over 200 million positions per second, defeating World Champion Garry Kasparov in a 1997 match, scoring 3.5-2.5 over six games.

This landmark achievement demonstrated machines could surpass human chess ability, yet *Deep Blue*'s specialized hardware prevented widespread software distribution. Subsequent research focused on improving gameplay while enabling broader accessibility. Recognizing evaluation function quality as paramount, researchers employed machine learning methods such as genetic algorithms to optimize evaluation parameters [8]. While improving performance, these approaches remained limited by human-defined positional factors.

With reinforcement learning demonstrating success in games and neural networks gaining popularity, engines such as *Giraffe* [9] implemented deep reinforcement learning in 2015, replacing Minimax with probability-based search to achieve International Master strength (the second-highest FIDE title). Despite progress, human mastery remained unconquered. DeepMind's 2016 application of reinforcement

learning with Monte Carlo Tree Search (MCTS) [10] to Go in *AlphaGO* [11] defeated the world Go champion, highlighting Go's greater complexity compared to chess. Subsequent chess applications struggled due to chess's tactical nature, where single moves can determine outcomes, presenting challenges for MCTS's probabilistic approach.

Addressing these difficulties, *DeepChess* [12] maintained Minimax while replacing evaluation functions with two neural network architectures: Deep Belief Networks (DBNs) [13] acting as unsupervised autoencoders for feature extraction, feeding into a siamese network [14] for supervised comparison learning. Trained on over 3 million chess positions labeled by game outcomes, this approach reached Grandmaster level and replicated advanced human choices, though human game limitations restricted further advancement.

In 2018, DeepMind returned to MCTS with enhancements and self-play reinforcement learning in *AlphaZero* [15]. Within hours of training, it surpassed human play and repeatedly defeated *Stockfish* [16], then the leading public engine. As an open-source engine designed for CPU versatility, *Stockfish* incorporated a neural network architecture (NNUE) [17] in its 2020 version, initially successful in Japanese Shogi, optimizing CPU calculations while maintaining hardware flexibility for mobile, online, and robotic applications.

Technological advances in chess engines, driven by neural networks and artificial intelligence, have profoundly transformed the chess landscape. These tools now surpass even renowned professionals, marking a notable achievement in chess history. Initially assisting coaches and analysts in game preparation, engines now propose moves of such sophistication that they challenge master-level comprehension. This evolution presents significant challenges for beginners and intermediate players who struggle to interpret engine recommendations due to their inherent complexity.

This context creates a need for moves better aligned with human comprehension levels [18], particularly in specific situations. Human-like moves, while not necessarily objectively optimal according to engines, remain theoretically sound and lower-risk. Aligning engine recommendations with human style could make learning more accessible and player progression more consistent throughout chess journeys.

Existing approaches typically address either board digitization using computer vision techniques or move prediction using machine learning models, treating these as independent problems. Moreover, most chess systems rely on engines such as *Stockfish* that optimize for the strongest move, disregarding whether suggestions align with human reasoning, creating a gap that is particularly relevant in educational and analytical contexts.

This work addresses both challenges within a unified framework, proposing an integrated pipeline that combines: (i) realistic synthetic data generation via photogrammetry for robust board digitization; (ii) a convolutional neural

network-based human-move prediction model trained on curated, style-specific datasets; and (iii) a hybrid decision mechanism in which the neural network acts as a pre-filter over candidate moves, while a traditional engine (Stockfish) is responsible for evaluating and validating the final move selection. This design is motivated by the fact that neural networks, due to their statistical nature, may produce plausible yet suboptimal or even erroneous moves, particularly in unseen scenarios, while the integration with Stockfish ensures that the final recommendation maintains a strong tactical and strategic quality. This design explicitly separates human-like move generation from tactical optimization, clarifying that the proposed system augments, rather than replaces, a chess engine. This work also investigates the impact of dataset homogenization (through opening selection and player rating filtering) on model convergence and predictive performance across different game phases.

To better position this work with respect to the literature, the main contributions and novelties are summarized in Section III. In brief, this work proposes a unified pipeline that integrates chessboard digitization and human-move prediction within a hybrid framework, combining data-driven models with engine-based validation. A structured comparison with prior work is presented in Section V-F, highlighting the research gaps addressed by the proposed approach.

II. OBJECTIVES

A. GENERAL OBJECTIVE

This project aims to develop an integrated system that combines chessboard digitization from physical images with human-like move prediction, providing players with accurate board recognition and strategically sound, interpretable move suggestions.

B. SPECIFIC OBJECTIVES

For chessboard digitization:

- Build a synthetic training database using photogrammetry-based 3D models and evaluate its effectiveness for piece classification via transfer learning.
- Develop and validate a pipeline capable of accurately identifying piece types, colors, and positions from real board images.

For human-like move prediction:

- Train CNN-based classifiers on curated human game data and assess the influence of dataset homogenization on model performance across game phases.
- Evaluate a hybrid approach that combines the prediction models with a chess engine to produce moves that balance human interpretability and tactical quality.

III. CONTRIBUTIONS

This research makes contributions at the intersection of chessboard digitization and human-like move prediction, addressing limitations found in prior work that typically treats these as independent problems:

- A photogrammetry-based pipeline for generating realistic 3D models of Staunton chess pieces and boards, producing a reusable synthetic image database with controlled variations in lighting, camera angles, and piece configurations.
- A transfer learning methodology that pre-trains a MobileNet V2 piece classifier on synthetic data and fine-tunes it on a small set of real images, achieving 99.78% test accuracy while substantially reducing dependence on real-world data collection.
- A CNN-based move prediction system trained on style-specific human game data, comprising a piece-to-move classifier and per-piece destination classifiers that jointly filter the candidate move space toward human-like patterns.
- A hybrid decision pipeline that integrates the human-move pre-filter with Stockfish evaluation, increasing the human-like nature of suggested moves by approximately 30% compared to unfiltered Stockfish output.
- An empirical analysis of the effect of dataset homogenization on model training stability and prediction accuracy, evaluated separately across the opening, middlegame, and endgame phases.

These contributions are detailed throughout the paper, with quantitative results and comparisons to prior work presented in Section VII. Beyond chess, the synthetic data generation methodology and the hybrid human-aware filtering mechanism are applicable to other domains where training data is scarce and interpretability of automated decisions is valued.

IV. RESEARCH QUESTIONS

This project seeks to answer fundamental questions that guide the investigation, addressing the technical challenges and opportunities at the intersection of computer vision, machine learning, and chess. Among these questions, the following stand out:

- Is there an advantage in using realistic 3D models generated through photogrammetry to create training image databases, and can these models reduce the need for large sets of real images?
- To what extent does transfer learning, using synthetic data generated by 3D models, maintain or improve system accuracy when applied to real objects?
- Is it possible to optimize move prediction by grouping game data according to player style or skill level, and what is the impact of this approach on model training homogeneity and quality?
- Is there a benefit in integrating the digitization and move suggestion models with a chess engine, such as Stockfish, to create a hybrid tool that combines strategic precision with human accessibility?

By exploring these questions, this project aims to contribute to the advancement of computer vision and machine learning applications in strategy games, highlighting both

the technical challenges and the practical and theoretical implications of the proposed solutions.

V. RELATED WORK

A. HISTORICAL EVOLUTION OF CHESS ARTIFICIAL INTELLIGENCE

The foundation of computer chess dates back to Claude Shannon's 1950 paper [2], which established two fundamental approaches: Type A strategies considering all moves to a fixed depth, and Type B strategies selectively exploring promising variations. This framework underpinned early systems such as Turing's Turochamp [4] and Berliner's Analyzer [19]. The 1970s witnessed the first computer chess tournaments, with programs such as Northwestern's CHESS 4.6 [6] achieving master-level performance through alpha-beta search improvements and specialized hardware.

IBM's Deep Blue project [7] represented a watershed moment, combining custom VLSI chess chips with parallel search algorithms to defeat World Champion Garry Kasparov in 1997. Deep Blue evaluated up to 200 million positions per second, yet its evaluation function remained human-crafted. This achievement spurred research into more efficient search methods, including null-move heuristic [20] and time and depth controls [21] that enhanced search quality without proportional increases in the examined nodes.

B. NEURAL NETWORK AND MACHINE LEARNING APPROACHES

Machine learning integration began with linear regression tuning of evaluation parameters [22], evolving to genetic algorithms for feature weighting [8]. The Giraffe Engine [9] pioneered deep reinforcement learning in chess, training through self-play to reach the International Master level. Its probabilistic search replacement for fixed-depth Minimax demonstrated promising results, though it fell short of the top traditional engines.

DeepMind's AlphaZero [15] revolutionized computer chess by combining deep neural networks with Monte Carlo Tree Search, learning entirely through self-play without human games. Its success against Stockfish highlighted the potential of learned evaluation functions over human-crafted ones. In response, the open-source community developed NNUE (Efficiently Updatable Neural Networks) [17], enabling neural network evaluation on consumer hardware while maintaining Stockfish's search efficiency.

Hybrid approaches such as DeepChess [12] employed deep belief networks and siamese architectures to compare positions, achieving Grandmaster-level play with more human-like characteristics. Concurrently, convolutional neural networks demonstrated strong move prediction capabilities [23], framing chess as a classification problem where the network selects among legal moves given board configurations.

Recent work explores hybrid designs: for instance, Kevanishvili and Iavich [24] introduced a hybrid

framework integrating Stockfish's evaluations with Maia-derived human-move features. They trained a CNN classifier that distinguishes engine-like vs. human-like play patterns, achieving a macro F1 of 0.93 (outperforming an engine-only baseline). These hybrid models suggest new ways to combine learning and traditional search.

C. COMPUTER VISION AND BOARD DIGITIZATION

Physical board digitization presents complementary challenges to algorithmic play. Early robotic systems used Digital Game Technology (DGT) boards [25] with embedded sensors, transmitting game states digitally [26]. While reliable, this approach required specialized equipment, limiting general applicability. Computer vision alternatives emerged with systems such as Gambit [27], using cameras to detect board geometry and piece presence, inferring piece identities from known starting positions.

ChessVision [28] divided digitization into board detection and piece classification stages, employing support vector machines for piece recognition. Limitations included manual dataset creation and substantial training image requirements. LiveChess2FEN [29] demonstrated improved performance with expanded datasets, yet generalization across diverse chess sets remained challenging. Masouris and van Gemert [30] proposed a fully end-to-end CNN model using the new ChessReD dataset (10,800 smartphone images), achieving 5.26% board recognition error (about 7 times better than previous methods).

Synthetic data generation through computer graphics [31] addressed dataset limitations by producing varied board positions and piece appearances. UnityChess created photo-realistic training imagery, while Karia et al. [32] developed complete digitization pipelines combining synthetic training with real-world prediction, enabling robust position recognition under varied conditions. For example, Wolflein et al. [33] created a large synthetic dataset and an end-to-end recognition system (RANSAC for board localization + CNNs for pieces), achieving only 0.23% error per square on the test set (99.77% accuracy) and 99.83% accuracy on a new chess set after fine-tuning with two images.

D. HUMAN-AI ALIGNMENT AND EDUCATIONAL APPLICATIONS

As chess engines surpassed human playing ability, research increasingly turned to the challenge of aligning engine recommendations with human comprehension. McIlroy et al. [18] investigated methods for identifying human-like moves that maintain strategic soundness while being more interpretable to players across skill levels. Educational applications have leveraged this approach to create training systems that explain suggestions in human-understandable terms rather than focusing exclusively on engine-optimal play.

Research in explainable artificial intelligence (AI) for games [34] has developed techniques for identifying the

strategic rationales behind moves, while studies [35] have examined the alignment between human and AI decision-making in chess through models that coherently capture human play styles across different skill levels. By introducing a skill-aware attention mechanism, their approach dynamically integrates players' strengths with positional representations, offering a unified framework that models how human strategies evolve with experience. These interdisciplinary efforts aim to bridge the gap between optimal play and pedagogical value, making advanced chess analysis accessible and beneficial across the skill spectrum.

Zhong et al. [36] presented a skill-group-specific n -gram framework, achieving up to 39.1% relative accuracy improvement over baselines. These works confirm that human move patterns can be systematically learned by skill level. Kevanishvili and Iavich [24] also applied a hybrid approach in this domain: they combined Stockfish and Maia features in a neural classifier to discriminate genuine human play from engine-optimized play, highlighting the benefits of mixed feature sets. Together, these approaches aim to make AI advice more aligned with human reasoning, benefiting educational tools and analysis.

Building on this foundation in human-AI alignment, recent work has explored advanced neural architectures specifically designed for this purpose. *Hari Sekar and Jin* [37] introduced a multitask transformer model that demonstrates significant potential for mimicking human decision-making patterns in chess. This approach represents a contemporary effort to bridge the gap between superhuman AI performance and human-like reasoning, creating systems that are more understandable and capable of effective collaboration with human players.

E. PHOTOGRAMMETRY AND 3D RECONSTRUCTION

Photogrammetry software such as 3DF Zephyr has become increasingly relevant for reconstructing 3D models from 2D images, with applications extending to chess piece and board reconstruction. Studies evaluating photogrammetry software accuracy [38] have shown that tools such as 3DF Zephyr can produce reasonably accurate 3D models, with best results achieved using ambient lighting conditions and references in the image. These techniques could potentially be applied to create detailed 3D chess models for both analysis and visualization purposes.

The 3DF Zephyr software [39] employs advanced algorithms including SIFT (Scale-Invariant Feature Transform) [40] and SURF (Speeded-Up Robust Features) [41] for feature detection, RANSAC (Random Sample Consensus) [42] for point matching, and structure-from-motion (SfM) approaches [43] for 3D reconstruction. Open-source alternatives, such as COLMAP [44] (a general-purpose SfM+MVS pipeline) and AliceVision's Meshroom, offer similar 3D reconstruction capabilities. These capabilities make it suitable for generating detailed 3D representations

of physical chess arrangements, potentially supporting more accurate digitization processes.

F. SYNTHESIS AND POSITIONING WITH RESPECT TO PRIOR WORK

In summary, prior works in the literature tend to address isolated aspects of the problem. Research in computer vision has focused primarily on board digitization, while studies in machine learning have explored human-move prediction from digital representations, and engine-based systems have concentrated on optimizing move selection for maximum performance.

However, these approaches are typically developed independently, without considering the integration between perception, decision modeling, and engine evaluation. This separation limits their applicability in scenarios that require both real-world interaction and human-centered interpretability.

As shown in Table 1, existing approaches typically focus on isolated components of the problem. In contrast, the proposed method is the only one, to the best of our knowledge, that integrates board digitization, human-move prediction, and engine-based validation within a single unified pipeline, while explicitly incorporating a human-aware filtering mechanism.

In contrast, our work proposes a unified pipeline that integrates chessboard digitization and human-move prediction within a single framework. Moreover, we introduce a human-aware pre-filtering mechanism that reorders the candidate move space, guiding engine evaluation toward more interpretable and human-like alternatives while preserving tactical soundness through Stockfish validation.

This integrated perspective not only bridges the gap between perception and decision-making but also contributes to ongoing efforts in human-AI alignment by emphasizing moves that are both strong and aligned with human reasoning patterns.

VI. MATERIALS AND METHODS

A. COMPUTATIONAL TOOLS AND HARDWARE RESOURCES

For the implementation of the methods and models developed in this project, a computer with the specifications detailed in Table 2 was used. These hardware configurations are fundamental for the performance of model training and image rendering tasks, allowing for an efficient evaluation of experiment execution times. Considering the specifications of the system used is crucial for correctly interpreting the results and comparing them with other methods or data processing systems.

In addition to the hardware, a variety of computational tools were employed in the development of this project:

- **Python:** The Python programming language was used as the foundation for implementing the methods and executing the experiments [45].

TABLE 1. Comparison of related work and research gaps addressed by the proposed method.

Work	Digitization	Human Move Prediction	Engine Integration	Human-Aware Filtering
ChessVision / LiveChess2FEN	✓	–	–	–
Maia [18]	–	✓	–	✓
AlphaZero / Stockfish	–	–	✓	–
Hybrid Classifiers [24]	–	✓	✓	✓
Proposed Work	✓	✓	✓	✓

TABLE 2. Technical specifications of the computer used.

Component	Specification
CPU	Ryzen 3800X
RAM	32 GB 3200 MHz
GPU	GeForce RTX 3070

- **PyTorch:** The PyTorch library was used for developing and training the neural network models [46].
- **OpenCV:** The OpenCV library was used for image processing [47].
- **Matplotlib:** For data and result visualization, the Matplotlib library was used to create graphs and visual representations [48].
- **3DF Zephyr:** To create realistic 3D models of the chess pieces, the photogrammetry software 3DF Zephyr was employed, enabling accurate three-dimensional reconstruction from photographs [49].
- **Blender:** This open-source software is widely used for 3D modeling and rendering [50].
- **Stockfish:** Assisted in the analysis and validation of chess moves, providing support for evaluating positions and moves with the precision of a widely recognized chess engine [16].
- **Python-Chess:** The Python-Chess library was used to manipulate chess games, allowing for the integration of moves with board positions and facilitating the generation of initial and final positions of the pieces [51].

These tools, combined with the described hardware, provided a robust environment for the development and execution of the methods and models addressed in this project.

B. SELECTION OF CHESS GAMES

The chess games used in this project were extracted from the public *Lichess* database [52], covering matches played between January and March 2024, totaling 187,372,089 games. Due to the wide variety of styles and moves present, filtering criteria were applied to build a more homogeneous dataset, aiming to reduce move variability and create a bias in terms of skill level and playing style. In this way, the model focuses on more defined patterns, facilitating training and enabling more consistent analyses.

For this filtering, the following criteria were applied:

- **Exclusion of bullet-style games:** Games with a total time of 1 minute per player were excluded, since the high speed of bullet games can introduce less accurate or irregular moves.
- **Player rating range between 1800 and 2300:** This rating range represents intermediate and advanced players, as highlighted in red in the distribution shown in Figure 1. This range is reasonably close to the mean value, up to 2σ , providing a dataset that avoids both very beginner and extremely advanced playing styles.
- **Exclusion of games ended due to time forfeits:** Games interrupted due to time tend to include forced or abrupt moves, deviating from the natural flow of the match.
- **Italian Opening:** Selection by opening aims to create greater consistency in initial positions, as the Italian Opening is one of the most studied and followed by intermediate to advanced players.

It is important to note that, as the dataset is derived from online platforms, it may contain games affected by engine-assisted play. While the applied filtering criteria, such as restricting the rating range and excluding very fast time controls, help reduce the likelihood of such cases, they do not fully eliminate this possibility. Consequently, some degree of noise related to non-human move patterns may still be present in the data. More robust mitigation strategies could be employed to further address this issue, such as filtering games involving flagged accounts, applying engine-correlation analysis to detect suspicious move patterns [53] or using neural networks to detect computer games [54].

Alternative strategies for organizing the dataset were also considered, such as grouping games based on similarity using clustering techniques [55]. However, due to the scale of the dataset and the high variability of chess positions, this approach proved computationally impractical. Therefore, a more controlled and efficient strategy was adopted, using opening-based filtering to approximate stylistic consistency while keeping the data processing pipeline tractable.

After applying these criteria, 278,692 games were selected, representing a significant sample for training. This number surpasses the 20,000 games used in previous studies, such as Oshri and Keren [23], providing a robust sample that improves the model's generalization. With the dataset structured as (Position, Move) pairs, a total of 18,305,849 unique positions was obtained. It is worth noting that, although the training set contains no repeated positions, the test games are drawn from a disjoint set of matches not seen

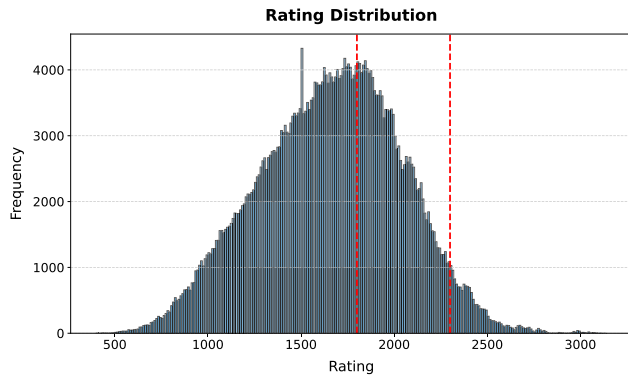


FIGURE 1. Distribution of player ratings for 200,000 games.

during training. Nevertheless, certain board configurations, particularly those arising in the early moves of structured openings such as the Italian, are likely to recur across training and test sets. This overlap is most pronounced in the opening phase and will be taken into account in the qualitative analysis of the results.

These criteria induce a bias in the model, making it more suitable for predicting typical moves of intermediate and advanced players who follow Italian Opening patterns. Conversely, the model will be less accurate in predicting moves from beginners or highly experienced masters, especially in the opening moves. Nevertheless, careful selection improves pattern detection and contributes to a more consistent game analysis, ensuring a solid base of positions and moves frequently observed in intermediate-level games.

C. DIGITIZATION OF THE CHESSBOARD

1) RENDERING TRAINING IMAGES FROM REALISTIC 3D MODELS

To create an extensive training image dataset with a realistic appearance, the process started with building 3D models of the chess pieces using photogrammetry in 3DF Zephyr [49]. This process was carried out from photographs of individual pieces. After generating the 3D models of the pieces and the board, the complete scene was assembled in Blender, where different piece positions on the board were configured based on the games described in the previous section. The images were then rendered from these scenarios, allowing for a variety of representations. This workflow was segmented into four main steps, as illustrated in Figure 2.

a: STEP 1: CAPTURING PHOTOS OF THE PIECES

The first step in digitizing chess pieces consists of capturing high-quality images, which are essential for creating the 3D models. To ensure that the images are suitable for the photogrammetry process, some fundamental conditions must be observed during capture:

- The object must be in focus, occupying a large portion of the image;
- High contrast between the object and the background;

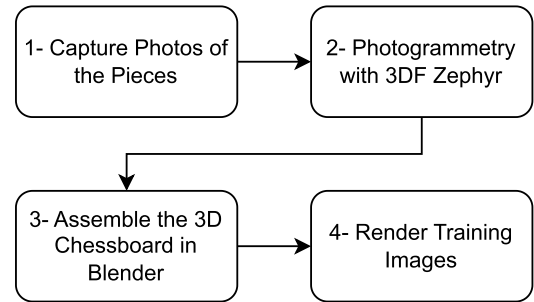


FIGURE 2. Diagram of the creation of synthetic training images.

- Adequate lighting, without excessive shadows;
- Obtaining images from all angles of the object;
- Use of a reference to identify the rotation of the piece.

These guidelines are crucial to ensure that the photogrammetry software can reconstruct the pieces in 3D with high accuracy. In Figures 3 and 4, the preparation of the environment for image capture can be observed. The pieces were positioned on a rotating base, and a smartphone camera was fixed to ensure consistency and stability during the process.

The free version of the photogrammetry software used allows up to 50 photographs per object, with good results obtained using approximately 36 images per piece, with a rotation of about 10 degrees between photographs. The base with geometric figures, clearly shown in Figure 4, was essential for allowing the software to correctly identify the piece's rotation and thus generate a complete 3D reconstruction. This care was especially important for pieces with geometric symmetry, such as the pawn, where correctly identifying the rotation is crucial to avoid distortions in the final model.

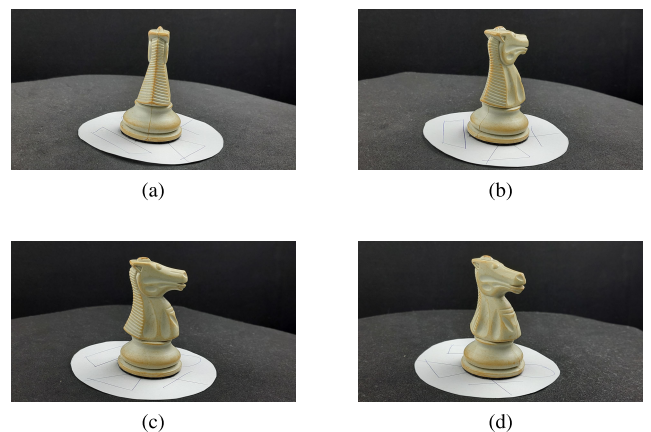


FIGURE 3. Images of the white knight for photogrammetry.

b: STEP 2: PERFORMING PHOTOGRAMMETRY

The second step of the process involves using 3DF Zephyr software to perform photogrammetry on the pieces. After loading all the images captured in Step 1, the next step is

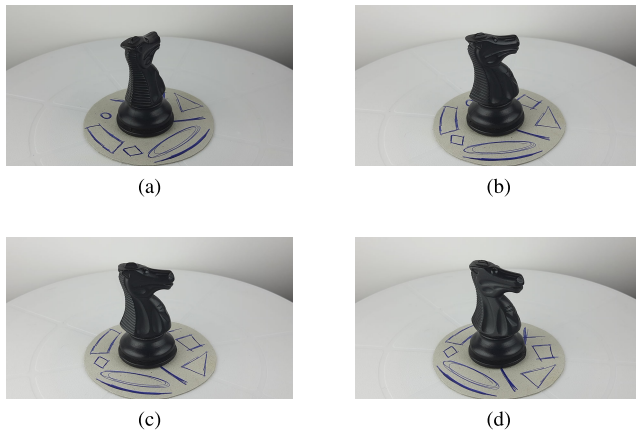


FIGURE 4. Images of the black knight for photogrammetry.

to mark, on one of the images, what corresponds to the background and what corresponds to the object (the chess piece). The software is able to generalize these markings to the remaining images, ensuring an accurate 3D reconstruction of the object. It is essential to configure the software to generate textures and carry out all processes at the highest possible quality in order to obtain a detailed final model.

After performing these adjustments and processing, 3DF Zephyr generates a 3D model of the piece, as illustrated in Figure 5. With this model, it can be exported to other software packages for further manipulation. In this project, the generated 3D model was exported to Blender, where the rendering and synthetic image generation steps were carried out.

This process was performed for 12 chess pieces, with one piece of each type for both white and black colors.

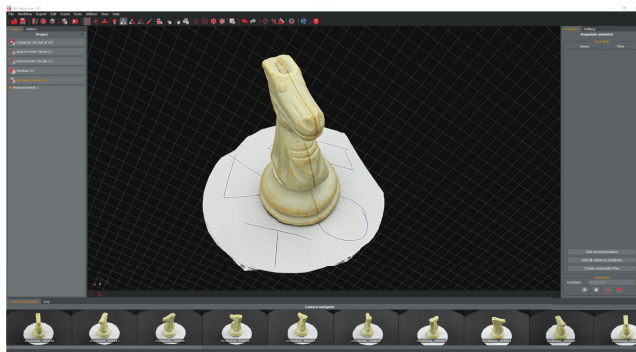


FIGURE 5. Reconstructed 3D object.

c: STEP 3: ASSEMBLING THE 3D CHESSBOARD

After generating and importing the 3D pieces into Blender, the polygons corresponding to the reference bases were removed. Next, a 3D scene was assembled in Blender, creating a complete chessboard, as shown in Figure 6. The board was designed using only a textured plane with a single photograph representing the board pattern.

In addition to constructing the board, it was necessary to configure the lighting and camera position in the 3D scene.

These settings were made to simulate the environment of capturing a real photograph of the board, similar to the initial digitization process of the pieces. The lighting was adjusted to ensure good visibility of the pieces and the board, and the camera position was adjusted to accurately reproduce the image capture angle.



FIGURE 6. 3D scene in Blender.

d: STEP 4: RENDERING IMAGES

With the 3D scene assembled, a Python script was implemented to edit the Blender file, changing the positions of the pieces according to the game positions obtained in Section VI-B, and automatically rendering an image that simulates a photograph of a real match. This process allowed the generation of 10,200 images, each representing a different configuration of the pieces on the board, ensuring a wide variety of angles and piece arrangements.

The use of 3D objects generated from real pieces enabled the creation of a large number of training images while maintaining a realistic visual appearance, as illustrated in Figure 7. This approach is essential for building a robust dataset capable of training the piece classification model with high accuracy.



FIGURE 7. Rendered image of the realistic 3D chessboard.

2) IDENTIFICATION AND DIVISION OF CHESSBOARD SQUARES

Dividing the chessboard into its 64 individual squares from an image requires the application of several image processing methods. This process is essential for isolating

each square, enabling precise subsequent analysis. The following describes each step of the procedure, as illustrated in the diagram shown in Figure 8.

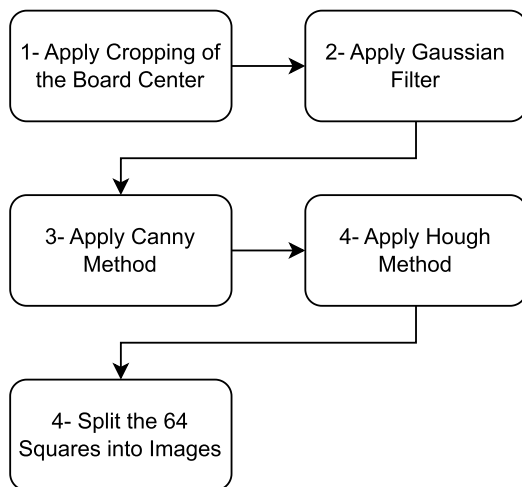


FIGURE 8. Diagram for dividing the chessboard squares.

The first step consists of manually cropping the image, considering that the camera and board will always be in fixed positions. This isolates the center of the board while discarding peripheral regions that are not relevant for the analysis. This cropping is important to reduce noise at the image edges and focus only on the area where the squares are located. Figure 9 (a) shows the initial result of the cropping, with the board centered and the region of interest clearly delineated.

After cropping, the next step is applying a Gaussian filter, as shown in Figure 9 (b). The Gaussian filter smooths the image by reducing abrupt pixel variations, helping to minimize small irregularities and smooth the edges. This process helps eliminate noise that could interfere with detecting the board lines and edges.

With the smoothed image, the next step is edge detection using the Canny operator [56]. The Canny algorithm is widely used for edge detection because it efficiently highlights regions of intensity transition precisely. Figure 9 (c) illustrates the detected edges, where the board lines become more clearly visible.

The following step involves applying the Hough algorithm [57], as demonstrated in Figure 9 (d). The Hough transform is used to detect straight lines in the image, corresponding to the horizontal and vertical lines that form the chessboard structure. Detecting these lines is critical for constructing the board grid.

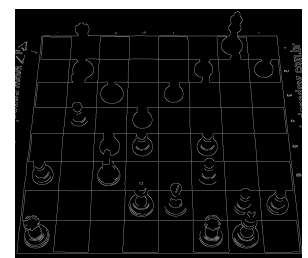
Once the horizontal and vertical lines are located, the next task is to calculate their intersections. The intersections represent the exact points where the square edges meet, allowing the board to be correctly segmented. Figure 9 (e) shows the intersections, highlighting the key points that delimit the squares on the board. These intersection points are crucial, as they form the basis for image segmentation.



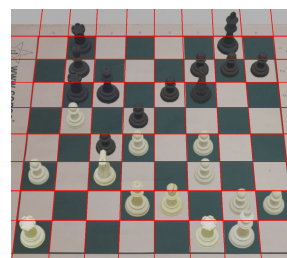
(a) Cropped Original Image



(b) Gaussian Filter



(c) Canny Method



(d) Hough Method



(e) Identified Intersections

FIGURE 9. Results for dividing the chessboard squares.

Finally, with the intersections of the horizontal and vertical lines identified, the board is segmented into 64 parts, each representing an individual square. This segmentation is performed based on the intersection positions, resulting in separate images for each square. The segmentation process allows each square to be treated as an isolated unit, which is fundamental for analyzing each space on the board. In this way, each piece is classified individually, and with all positions and corresponding classifications, it is possible to reconstruct the complete board configuration.

3) CLASSIFIER TRAINING

a: PRE-TRAINING WITH 3D MODELS

After digitizing the board and segmenting the images of each square, the next step is to train a classifier. For this purpose,

a dataset composed of 80,000 images of individual pieces was used, extracted from the 10,200 chessboard images generated from realistic 3D models, with 80% allocated for training and 20% for validation. During the collection of these images, only pawns were limited to two pieces per position to mitigate class imbalance. However, an imbalance is still noticeable in the dataset, with a significantly higher number of pawns compared to other pieces, and other pieces such as queens and kings with fewer examples, as shown in Tables 3 and 4.

Class imbalance is a well-known challenge in machine learning, particularly in scenarios where certain classes are underrepresented, which may lead to biased models and misleading performance if not properly addressed. As discussed by Abdulrazaq [58], models trained on imbalanced data may achieve high overall accuracy while failing to correctly learn minority classes, highlighting the importance of appropriate data handling and evaluation strategies.

TABLE 3. Total distribution of synthetic white pieces.

Piece	Quantity
Pawn	15,702
Bishop	3,475
Knight	3,712
Rook	4,529
Queen	2,221
King	2,357

Among white pieces, pawns account for approximately 49% of the dataset, while queens and kings together represent less than 15%, reflecting their lower frequency on the board. A similar distribution is observed for black pieces (Table 4), confirming that the imbalance is consistent across colors.

TABLE 4. Total distribution of synthetic black pieces.

Piece	Quantity
Pawn	15,619
Bishop	3,460
Knight	3,867
Rook	4,459
Queen	2,243
King	2,356

This consistent imbalance across both colors, where minority classes such as queens and kings contain fewer than 2,400 samples each, directly motivates the sampling strategy described below.

To handle this imbalance, taking advantage of the relatively large dataset, a *Weighted Random Sampler* was used. This method limited the number of images processed per epoch to 40,000, ensuring that each epoch used a different subset of training data, while maintaining balance between classes. This strategy is aligned with commonly adopted techniques for mitigating imbalance, such as controlled sampling and reweighting, which aim to reduce bias toward majority classes while preserving dataset diversity. This approach

helped improve training quality and reduce the impact of class imbalance during the 20 training epochs.

The architecture chosen for the classifier was *MobileNet V2*, known for its computational efficiency and accuracy in classification tasks on low-resource devices. To adapt the network to the specific context of the chessboard, the last classification block was modified to identify 12 classes: chess pieces in white and black (king, queen, bishop, knight, rook, and pawn). Thus, the model is capable of recognizing not only the presence of a piece in a square but also its type and color.

To reduce the processing of the board squares, a pre-selection step was implemented before sending the image to the classifier. In this step, a Gaussian filter is applied to smooth the image, and adaptive thresholding with a Gaussian-weighted sum converts the image to black and white. Based on this binarization, the ratio between white and black pixels is calculated. When this ratio is extremely low, indicating that the square is visually homogeneous, it is concluded that no piece is present, and the square image is discarded, saving computational resources.

This procedure allows the classifier to focus only on squares that actually contain pieces, reducing computational load and increasing model accuracy.

b: FINE-TUNING WITH REAL PHOTOS

To increase the classifier's accuracy in real-world conditions and approximate actual chess game scenarios, fine-tuning was performed with images captured directly from physical boards (Figure 10), allowing all network weights to be updated without freezing any layers. Images were captured for every move in three complete games, resulting in a total of 129 photographs. Each of these images was segmented following the previously described procedure, totaling 3,027 images of squares occupied by pieces.



FIGURE 10. Image of a real chessboard.

The dataset was split into 50% for training, 20% for validation, and 30% for testing, with the latter used exclusively after pre-training convergence and fine-tuning. This split was intentionally designed to limit the amount of data available for training, in order to evaluate whether pre-training could achieve good results even with a reduced

dataset. This approach addresses a limitation highlighted in similar methods [32], which, although achieving good results, indicated the need for a larger dataset to improve performance.

Due to the large disproportion of pawn images relative to other pieces, a balancing strategy was required. Initially, the same approach with a *Weighted Random Sampler* was used, but it did not yield satisfactory results. This likely occurred due to the high disproportion in a small dataset, as in this case, no limit was applied to the collection of pawn images, resulting in over 2,000 images of this class. The variability in the dataset generated by the sampler each epoch hindered training and made model convergence difficult.

Consequently, part of the pawn images in the training set was removed, significantly reducing their quantity. After this adjustment, the final set consisted of 976 images for training, with the piece distribution shown in Tables 5 and 6, 605 for validation, and 909 for testing. To train minimally with real images, fine-tuning consisted of 4 epochs. It should be noted that the real-world evaluation was conducted using a single physical chess set captured under controlled lighting conditions. While this setup is sufficient to demonstrate the viability of the synthetic-to-real transfer approach, it represents a limitation in terms of real-world breadth, as variations in piece style, board material, lighting environment, and camera angle found in unconstrained settings were not fully covered by the current digitization experiments.

TABLE 5. Distribution of real white pieces in the training set.

Piece	Quantity
Pawn	120
Bishop	85
Knight	71
Rook	117
Queen	34
King	52

The real training set is substantially smaller than the synthetic one, with the least represented classes, queen and king, containing only 34 and 52 samples respectively for white pieces. A comparable scarcity is observed for black pieces (Table 6), which reinforces the need for pre-training with synthetic data to compensate for this limitation.

TABLE 6. Distribution of real black pieces in the training set.

Piece	Quantity
Pawn	111
Bishop	103
Knight	94
Rook	99
Queen	32
King	58

With queens and kings again accounting for the fewest samples, 32 and 58 for black pieces, the real dataset alone

would be insufficient to train a reliable classifier, directly motivating the transfer learning approach evaluated in the following section.

In this stage, two models were trained for comparison: one with initial weights loaded from the previous training with synthetic images, and another *MobileNet V2* model initialized with random weights, trained only with real photographs. The model fine-tuned from the previously trained synthetic weights had the advantage of starting with prior knowledge about chess pieces, allowing it to better adjust parameters according to the specific characteristics of real images, such as lighting, board textures, and small piece details. The other model, trained with the same real images, served as a comparison point to evaluate the effectiveness of pre-training with synthetic images.

D. MOVE PREDICTION

This section is dedicated to the development of neural network models for predicting moves in chess games, based on real data from human-played games. As illustrated in the diagram in Figure 11, the process essentially involves three main steps after game selection, previously detailed in Section VI-B. The proposed method employs convolutional neural networks (CNNs) to predict moves, as presented in [23], using a large amount of data. Unlike approaches that aim to identify the best possible moves [59], the focus here is on capturing characteristic human moves [18] and using an evaluation tool to suggest the best among those moves [60].

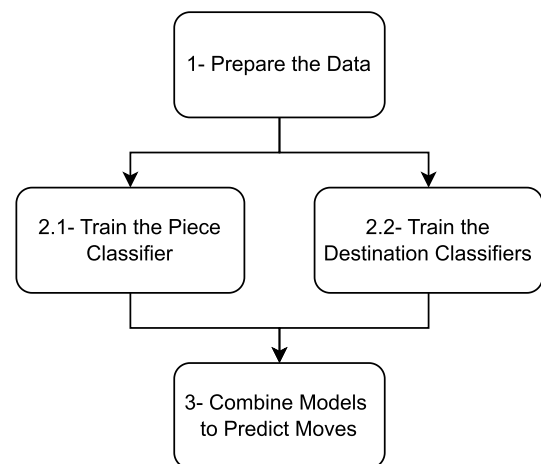


FIGURE 11. Diagram for move prediction.

Starting with the first step, this section details the data preparation process, which includes selecting the positions from the previously chosen games and vectorizing these positions. This step is essential to convert the games into a numerical structure suitable for neural network processing.

Next, the construction of convolutional neural network architectures is addressed, specifying the parameters and layers used in the model, as well as the training techniques applied to optimize predictive performance. It is important to

highlight that these models are independent, both in training and inference. One model is responsible for classifying the most likely piece to make a move, while other models are dedicated to identifying the most probable target square for each piece. The results of both models are combined to form a probability list used in the subsequent step.

Finally, the section concludes with an explanation of the integration and combination of both models, demonstrating how they are used to create a system capable of predicting the best moves in chess games while remaining understandable to human players.

1) DATA PREPARATION

a: POSITION SELECTION

For the move prediction step, the critical information used consisted of the board positions and the move executed from each position. All positions from the games were vectorized and organized into pairs of (position, move). Each pair thus reflects the board state immediately before a move and the corresponding executed move.

If the same board position appeared in different games with different moves, the most frequently played move was kept. In cases where two moves had the same frequency, one was chosen at random. To facilitate model training, moves for white pieces and black pieces were separated into different datasets. In this project, the focus is on white pieces, as the results for black pieces were essentially equivalent and the number of (position, move) pairs was similar. The final dataset consisted of 8,616,841 pairs for white pieces, providing a broad basis for model training and evaluation.

This data preparation ensures that the model receives consistent and balanced information, allowing it to learn recurring patterns and make predictions based on a diversity of game situations.

b: BOARD VECTORIZATION

For neural network models to process and analyze the positions of pieces on a chessboard, it is necessary to transform the piece arrangement into a suitable numerical structure. Board vectorization converts the position of each piece into a three-dimensional matrix representing the game state, facilitating the use of convolutional networks to identify patterns and predict moves.

The chosen representation for the board is a binary matrix of size $8 \times 8 \times 12$. The first two dimensions (8×8) correspond to the board structure, which contains 64 squares arranged in 8 rows and 8 columns. The third dimension, with depth 12, represents the different piece classes and the two possible colors in chess.

Each depth layer in this matrix represents the presence or absence of a specific piece and its color:

- The first six layers represent white pieces: pawn, knight, bishop, rook, queen, and king.
- The next six layers represent black pieces, in the same order.

This binary structure is filled such that a cell contains the value 1 if the piece corresponding to that layer is present in the respective board position and 0 otherwise. For example, if a white knight is on position *d4* on the board, the value 1 is recorded in the cell corresponding to *d4* in the layer assigned to the white knight. If a position is empty, all values in the 12 layers for that cell are 0.

The resulting matrix is therefore a complete binary encoding of the board position, where the spatial distribution of the pieces is mapped clearly and directly. This representation provides a compact but informative data structure that allows the neural network model to learn piece patterns and predict possible moves from real game data.

2) CLASSIFIERS

a: PIECE-TO-MOVE CLASSIFIER

For the move prediction process, a convolutional neural network model was developed, specifically designed to take the board state as input and predict which piece is most likely to be moved. The architecture was carefully designed to process the spatial structure of the board, where the input consists of a three-dimensional matrix $8 \times 8 \times 12$ representing the position and type of each piece.

The network is divided into two main parts: the convolutional layers, responsible for feature extraction, and the fully connected layers, which perform the final classification.

The first part of the model consists of a series of convolutional layers that capture and refine the features of the pieces on the board. The detailed structure of these layers is as follows:

- **First Convolutional Layer:** A convolutional layer with 32 filters of size 3×3 , followed by batch normalization and a ReLU activation function to ensure consistent and robust feature extraction.
- **Second Convolutional Layer:** This layer increases the number of filters to 64, maintaining the same filter size, again followed by batch normalization and ReLU. A 2×2 max pooling operation is then applied, reducing the feature map resolution by half while preserving essential information.
- **Third and Fourth Convolutional Layers:** The third layer expands to 128 filters and the fourth to 256 filters, both with 3×3 convolution operations, batch normalization, and ReLU, followed by another max pooling operation. This gradual increase in filters allows the network to capture more complex features.
- **Fifth and Sixth Convolutional Layers:** Both layers have 512 filters with a filter size of 3×3 . The last convolutional layer is followed by an additional max pooling, preparing the network for the classification phase.

After convolutional feature extraction, the network passes the data to the fully connected layers:

- **First Layer:** A 25% dropout is applied to prevent overfitting. This is followed by a fully connected layer

with 1024 units and batch normalization to combine the extracted features.

- **Second Layer:** Similarly, a fully connected layer with 512 units is used, with dropout, batch normalization, and ReLU, preparing for final classification.
- **Output Layer:** The final layer has 6 units, one for each piece type (king, queen, bishop, knight, rook, and pawn), indicating the probability of each type of piece being moved.

To ensure that the model generalizes well to unseen data, 80% of the dataset was used for training and 20% for validation. This split allows the effectiveness of the classifier to be evaluated throughout the training epochs and enables parameter adjustment to maximize accuracy.

b: PIECE DESTINATION CLASSIFIER

The next model developed is a classifier aimed at predicting the destination square of a selected piece, with a neural network trained for each individual piece type. The goal is that, upon receiving the current board state, the model can predict which square the selected piece should move to, according to patterns observed in real games.

The neural network architecture for destination square prediction has a configuration similar to the piece classifier model, with an adaptation to better match the current output. As in the piece classifier, each neural network processes the board position as a three-dimensional tensor of $8 \times 8 \times 12$, but the output layer has been adapted to match the number of squares (64) a piece can potentially occupy. The final layer of the classifier therefore has 64 neurons, each representing a square on the board and the probability that the piece will move to that position.

For training each model, an 80% training and 20% validation split was used, as applied in the previous classifier. Each network was trained with a dataset specific to its corresponding piece, with the number of position-move pairs shown in Table 7.

TABLE 7. Distribution of moves for each piece.

Piece	Quantity
Pawn	2,275,272
Bishop	1,480,515
Knight	1,279,235
Rook	1,249,056
Queen	1,083,428
King	969,121

The distribution reflects natural human play patterns: pawns are the most frequently moved pieces, particularly in the opening and middlegame, resulting in over 2.27 million samples, approximately 2.3 times more than the king, which is typically moved sparingly to preserve safety. This imbalance across piece-specific datasets means that destination classifiers for more active pieces, such as pawns and bishops, benefit from richer training signal, while those for less frequently moved pieces, such as the king and queen,

face a more limited set of examples from which to generalize movement patterns.

This configuration allows each model to learn the movement patterns of its piece in a specialized manner, ensuring more accurate movement predictions and increasing the overall effectiveness of the move prediction system.

3) MODEL COMBINATION AND INTEGRATION WITH STOCKFISH

This section presents the approach to combine the developed classification models and integrate them with the Stockfish engine, creating an efficient pipeline for move prediction in chess games (Figure 12). The central idea is to use the piece and destination classifiers to predict plausible human moves and then employ Stockfish as a final filter to ensure the quality of the chosen move.

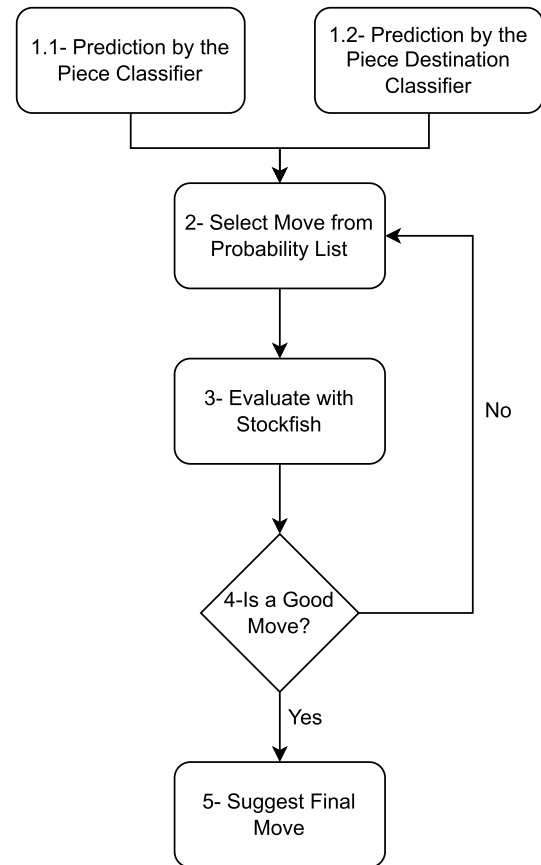


FIGURE 12. Diagram for evaluating predicted moves.

As illustrated in Figure 12, once a candidate move is selected from the probability list, Stockfish evaluates the resulting position in centipawns. A move is considered acceptable if the evaluation remains above 0 centipawns, that is, if the position is not assessed as disadvantageous for the moving side. If the move fails this criterion, the next candidate in the ranked list is evaluated, and the process repeats until an acceptable move is found.

The generation of the move probability list combines the predictions from the two classifiers, considering that each class result is a probability distribution generated by the *softmax* function. The piece classifier provides probabilities for each piece being moved, while the destination classifier provides probabilities for each possible target square.

The final probability of a specific move is obtained by combining the probabilities provided by the two distinct classifiers. Since the system independently predicts both the piece to be moved and its destination square, a mechanism is required to combine these two sources of information into a single move-level score. For example, suppose the piece classifier assigns a 90% probability for a pawn to be moved, while the destination classifier indicates a 70% probability for that pawn to move to square “e4”, distributing the remainder among other squares. In this scenario, the final probability of the move “pawn to e4” is calculated as the arithmetic mean of these two estimates (Equation 1), resulting in $P_{\text{move}} = 80\%$.

$$P_{\text{move}} = \frac{P_{\text{piece}} + P_{\text{destination}}}{2} \quad (1)$$

An initial approach explored in this work followed a sequential strategy, where the system first selected the most probable piece and then predicted its destination square. However, this formulation led to inferior experimental results and introduced a key limitation: it constrained the generation of Top- k candidate moves, since all candidates were conditioned on a single selected piece, reducing diversity and coverage of plausible alternatives.

Although the most intuitive approach would be to first select the piece to move and then select the destination square, assigning equal weight to both classifiers, regardless of order, provided the best experimental results. In particular, combining both predictions through an arithmetic mean enabled the generation of a unified ranking of candidate moves across different pieces, allowing a more flexible and expressive Top- k move list. This calculation is repeated for all possible moves, combining the predictions of the piece classifier with the destination predictions. The final move list and their probabilities are sorted in descending order, and the top moves are then analyzed by the Stockfish engine to validate their quality and feasibility within the context of the game.

It is worth noting again that the classification results are independent, allowing an interaction that subverts the conventional flow of first predicting the most likely piece to move and then predicting its destination square. Experimental tests showed that prioritizing all moves of the most probable piece before considering moves of the second most probable piece led to inferior performance and higher computational cost. Observations indicated that interleaving moves from different pieces in the probability ranking was more efficient. While the use of an arithmetic mean may be considered heuristic, it provided a good trade-off between simplicity, numerical stability, and empirical performance. More sophisticated alternatives, such as product-based combinations

with renormalization or learned fusion strategies, were not explored in depth in this work and remain as potential directions for future improvements.

The choice of arithmetic mean as the fusion operator implicitly assumes that the piece and destination probabilities are on comparable scales and contribute equally to the final move score. This assumption may not hold in all game phases. For instance, during the opening, piece selection may carry less uncertainty than destination prediction, suggesting that a phase-adaptive or learned weighting scheme could yield improvements.

Although the neural network models are designed to predict plausible human moves, some situations may result in impossible or suboptimal moves in the context of a game. To handle these cases, the Stockfish engine is used as a refinement step in the pipeline.

The complete process flow is as follows:

1) **Move Prediction by Neural Network:**

- The piece classifier predicts the most likely piece to move.
- The corresponding destination classifier predicts the square to which this piece should move.

2) **Move Validation with Stockfish:**

- The move predicted by the neural network is sent to Stockfish.
- Stockfish evaluates whether the move is legal and assesses its quality.

3) **Replacement of Improper Moves:**

- If the predicted move is illegal or deemed poor by Stockfish, the next move predicted by the neural network is evaluated.
- This process repeats until a valid and acceptable-quality move is found. As all possible moves will be analyzed, only in a different order than conventional Stockfish evaluation, there is no risk of failing to converge to a valid move.

4) **Selection of the Best Move:**

- Among the moves predicted by the neural networks, Stockfish selects the one representing the best possible move for the current position.

a: EVALUATION PROTOCOL FOR HUMAN-LIKENESS AND MOVE VALIDATION

The main objective of this pipeline is to combine the neural networks' ability to predict plausible human moves with Stockfish's computational strength to identify the best possible move. In this way, the system filters moves that would be highly unlikely for a human player while using Stockfish as a validation and refinement mechanism, ensuring the overall game strategy remains at a high level.

In this work, the training dataset was curated based on the Italian Opening in order to homogenize the move distribution, capture a more consistent playing style, and improve the model's ability to learn human-like patterns from a specific group of players. This approach seeks a balance between

realistic predictions and effective moves, creating a robust system for analyzing and predicting moves in chess games.

The final validation of the pipeline is performed on an independent test set containing 24,000 samples, all extracted from games that were not used during training or validation. These samples are divided into 6,000 positions for each of four openings: Italian Opening, Ruy Lopez, Sicilian Defense, and Queen's Gambit. This design makes it possible to evaluate the proposed system on the same opening used to build the training dataset, as well as on other openings, allowing a broader analysis of generalization and comparison with Stockfish.

Human-likeness is operationally defined as the ability of the system to predict a move that belongs to the set of moves actually played by human players in the same position. For example, if a given position X appears in the test set and, across all occurrences in the database, human players selected only moves y or z from that position, then a prediction is considered human-like only if the system suggests either y or z .

This structuring of the test set allows evaluation of the pipeline's effectiveness in different game contexts, providing a more detailed analysis of model performance throughout a chess match. The evaluation is carried out separately for three stages of the game:

- **Opening:** Considering the first 15 moves of each game, with 2,000 samples selected to represent typical strategies and patterns at this stage.
- **Middlegame:** Covering moves 16 through 30, also with 2,000 samples, where tactical complexity and piece interactions generally increase.
- **Endgame:** Including positions after the 30th move, with another 2,000 samples, reflecting scenarios where the number of pieces on the board is reduced and moves become more focused on completing the game.

Additionally, combining predictions made by the neural networks with Stockfish validation aims to capture both the plausibility of human moves and the strategic quality, while also acting as a safeguard against statistical errors from the neural network, mitigating hallucinated or highly suboptimal moves in previously unseen scenarios, providing a system applicable for both game analysis and human player training.

E. ASSUMPTIONS, TRAINING PROCEDURE, AND ROBUSTNESS CONSIDERATIONS

The proposed framework relies on several assumptions that are important for interpreting the results. First, the human-move prediction and piece classification models assume that the training datasets are representative of real-world conditions. This is partially enforced through the use of synthetic data generated from realistic 3D models, as well as dataset curation strategies such as piece balancing constraints and controlled sampling.

Additionally, the move prediction component adopts a factorized representation of moves, where origin and destination squares are modeled independently and later combined.

While this simplifies the learning process and reduces the output space, it may not fully capture dependencies between origin and destination, which is considered a limitation of the current approach. In particular, a joint representation that conditions the destination on the selected origin square could in principle model piece-specific movement constraints more faithfully, at the cost of a significantly larger output space and increased training complexity. This trade-off motivates the factorized design adopted here, while acknowledging its representational limitations.

The hybrid pipeline further assumes that Stockfish provides a reliable evaluation of tactical correctness. In this context, the neural network acts as a human-like prior, while Stockfish serves as a validation and refinement mechanism, ensuring that selected moves remain strategically sound.

From a training perspective, the piece classification model was implemented as a convolutional neural network trained using mini-batch stochastic gradient descent. The architecture consists of convolutional layers with non-linear activation functions (ReLU) and pooling operations, followed by fully connected layers for classification.

Training was performed using a batch size of 64 and optimized using the Adam optimizer, with an initial learning rate of 1×10^{-3} and a weight decay of 1×10^{-5} to improve generalization. A cross-entropy loss function was used for supervision. The learning rate was kept constant during training, and the model was trained for 20 epochs. To ensure clarity in the experimental setup, the primary training parameters are summarized in Table 8.

TABLE 8. Summary of experimental hyperparameters for neural models.

Hyperparameter	Value
Optimizer	Adam
Initial Learning Rate	1×10^{-3}
Weight Decay	1×10^{-5}
Batch Size	64
Training Epochs	20
Loss Function	Cross-Entropy

To mitigate class imbalance, a Weighted Random Sampler was employed, ensuring a more uniform distribution of classes within each batch. Each epoch processed approximately 40,000 samples, with stochastic sampling allowing different subsets of the dataset to be used across epochs.

To improve robustness, the training procedure incorporates stochastic sampling at each epoch, reducing overfitting and promoting better generalization across classes. The use of weight decay and balanced sampling further contributes to stabilizing the training process.

Although a full sensitivity analysis is beyond the scope of this work, we note that model performance may vary with respect to hyperparameters such as learning rate, batch size, and sampling strategy, as well as dataset composition. These aspects represent important directions for future investigation.

F. BOARD DIGITIZATION PIPELINE

Figure 13 provides a unified vision of the proposed framework. It illustrates the transition from the physical domain (photogrammetry and image capture) to the digital domain (classification and FEN generation), and finally to the decision domain (move prediction).

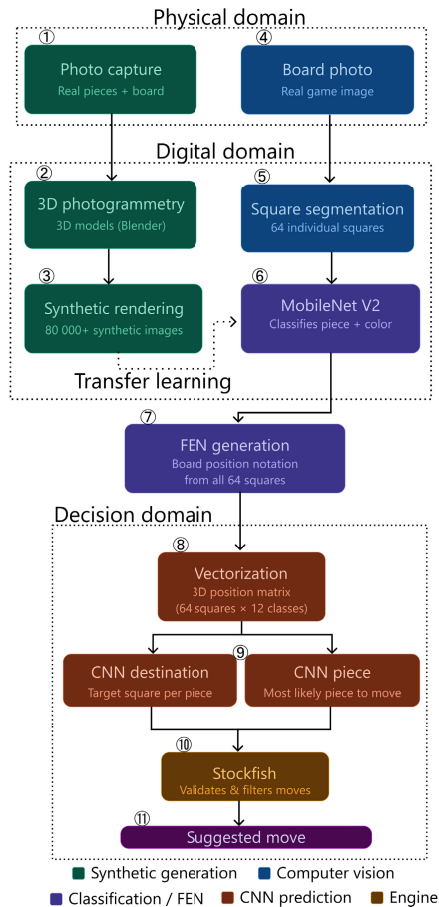


FIGURE 13. End-to-end system overview: integration of the digitization module (computer vision) and the move prediction module (deep learning).

The digitization workflow, as depicted in Figure 2, relies on synthetic-to-real transfer learning. This four-step process mitigates the common “data bottleneck” in computer vision by using the rendered 3D models from Figure 6 as a robust prior for the real-world classifier.

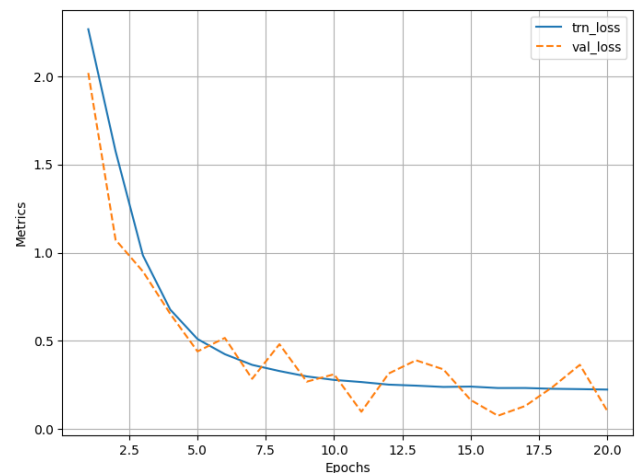
VII. RESULTS

A. BOARD DIGITIZATION RESULTS

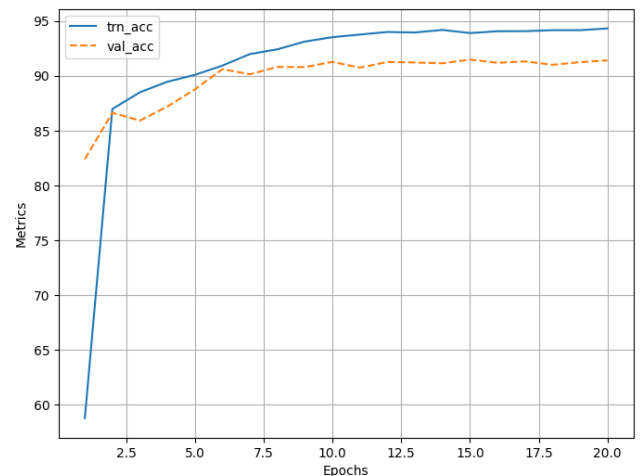
This section presents the results obtained during the board digitization process. To achieve the goal of identifying the pieces present on each square of the board, the model was initially pre-trained on a large set of synthetic images, followed by fine-tuning using real images, and finally its performance was compared with a model trained exclusively on real images, without pre-training.

1) CLASSIFIER PRE-TRAINING

The progress of the piece classifier pre-training is illustrated in Figure 14. Noticeable fluctuations can be observed, especially in the model’s performance on the validation set. These fluctuations can be attributed to the randomness introduced by the *Weighted Random Sampler*, which selects different sample distributions at each epoch to handle class imbalance. Although this strategy is effective in ensuring that the model is trained on a balanced dataset over epochs, it can also introduce variability in validation performance, as the evaluated samples may vary significantly in complexity and representativeness.



(a) Loss function during training



(b) Accuracy during training

FIGURE 14. Metrics evaluated during the piece classifier training.

Despite these fluctuations, the model shows an overall trend of convergence, indicating that the pre-training was able to capture the essential characteristics of the chess pieces from the synthetic dataset. A more detailed analysis suggests that the randomness controlled by the sampler, although challenging, positively contributes to the model’s

robustness by preventing it from becoming overly dependent on a specific subset of data.

The final training outcome achieved an accuracy of 94.3% on the training set and 91.4% on the validation set. While these metrics are promising, a detailed analysis of the confusion matrix, shown in Figure 15, reveals important aspects of the model’s performance.

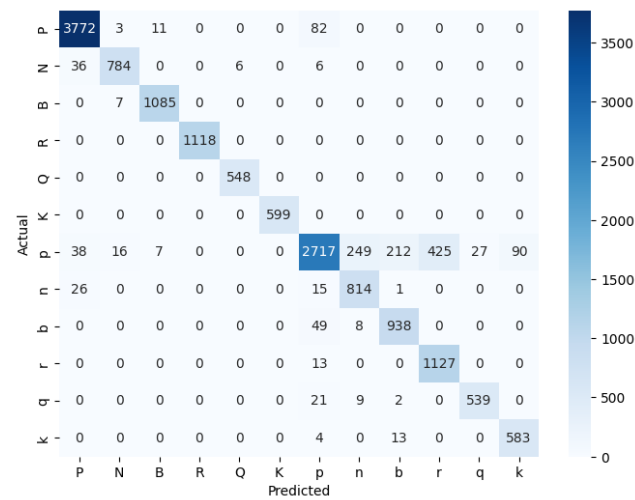


FIGURE 15. Confusion matrix.

The confusion matrix highlights that most of the classifier errors occur with black pieces, with a high frequency of misclassifications for black pawns. Although the random sampler significantly mitigated the effect of class imbalance for white pieces, resulting in few errors of this type, the performance on black pieces still presents challenges. This behavior may be related to the difficulty of the model in capturing the details of dark pieces, due to their subtler visual characteristics.

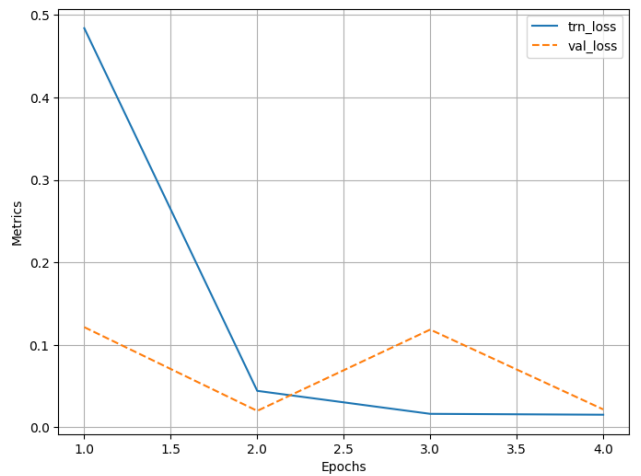
Additionally, the model demonstrated higher accuracy for classes such as white queens, bishops, and rooks, with a clear reduction in misclassification among these classes. This difference suggests that white pieces, with higher contrast, facilitate the extraction of discriminative features by the model.

Thus, the classifier pre-training achieved satisfactory results in terms of overall accuracy, while also highlighting limitations related to the recognition of black pieces.

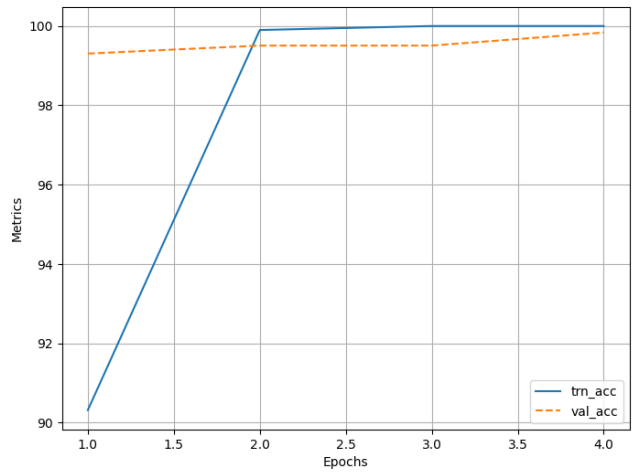
2) MODEL REFINEMENT WITH REAL IMAGES

After pre-training the model on a dataset of 80,000 synthetic images, refinement was performed using real images captured from complete chessboards. The training results are shown in Figure 16. This refinement was essential to adapt the model to the reality of captured images, as the domain of real images presents visual characteristics distinct from the synthetic images used in pre-training.

The refinement demonstrated promising performance metrics, achieving an accuracy of 100% on the training



(a) Loss function during training



(b) Accuracy during training

FIGURE 16. Metrics evaluated during model refinement with real images.

set, 99.84% on the validation set, and 99.78% on the test set. These results highlight the effectiveness of the domain adaptation process, allowing the model to generalize well to real data after only a few epochs of training.

Analysis of the confusion matrix for the refined model, shown in Figure 17, demonstrates a significant improvement in the classification of black pieces compared to the pre-trained-only model. Although some errors remain concentrated among black pieces, especially between pawns and bishops, the refinement drastically reduced the confusion issues observed previously. This improvement can be attributed to the model’s adaptation to real visual characteristics, such as subtle color tones and texture, which are often less detailed in dark pieces due to photogrammetry limitations.

Additionally, a test was conducted using the pre-trained model directly, without refinement. In this case, the accuracy was only 59.19%, highlighting that the transition from the synthetic to the real domain is not trivial. This reinforces

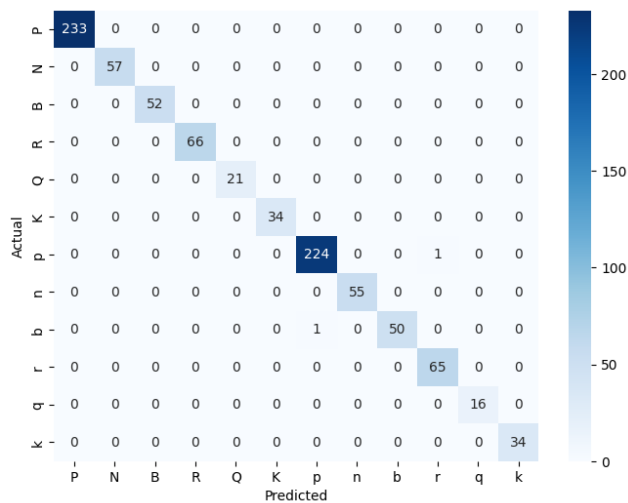


FIGURE 17. Confusion matrix of the classifier after refinement with real images.

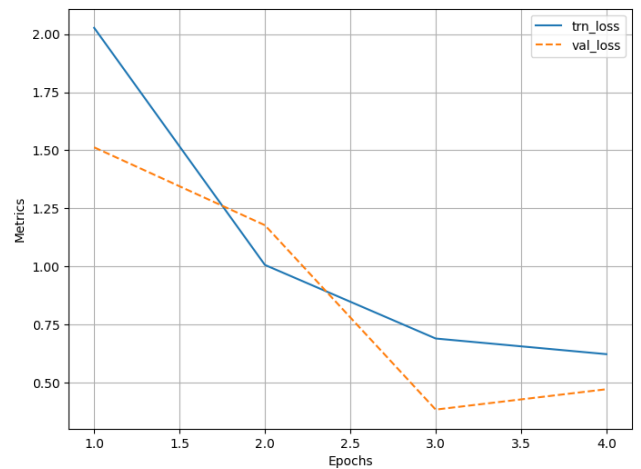
the importance of refinement to adjust the model to the new domain. The fine-tuning, although rapid, successfully aligned the model to the particularities of real images, resulting in significantly superior performance.

3) COMPARISON WITH MODEL TRAINED EXCLUSIVELY ON REAL IMAGES

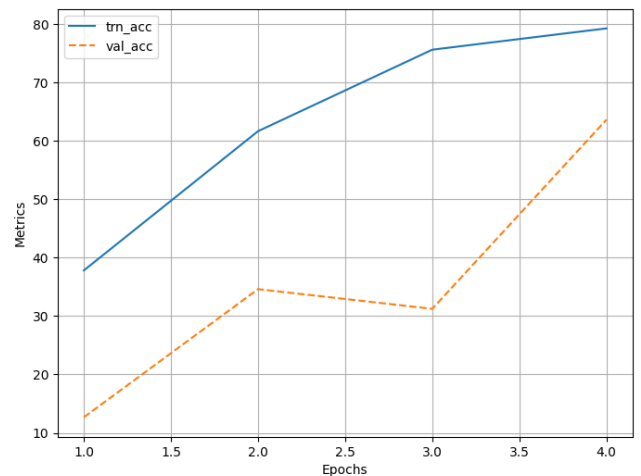
For a more in-depth comparative analysis, the model was trained using only real images of complete chessboards, without any pre-training with synthetic images. The results of this experiment highlight the limitations of this approach. The learning curve exhibited considerable inconsistency, as illustrated in Figure 18, with significant fluctuations in the loss function and accuracy, especially on the validation set. This instability, combined with a considerably longer convergence time, compromised the overall performance of the model.

The model trained solely on real images achieved an accuracy of 79.24% on the training set, 63.62% on the validation set, and 65.46% on the test set. These values are significantly lower than those obtained with the model that used pre-training with synthetic images. Additionally, the results showed a strong tendency toward overfitting, as the training set accuracy was substantially higher than that observed on the validation and test sets.

Analysis of the confusion matrix (Figure 19) reveals significant classification errors. White pieces, in particular, were poorly classified, as well as pieces with fewer representations in the training set, such as queens. On the other hand, the model performed better at classifying pawns, likely due to the larger number of samples available for this class. Regarding black pieces, considerable confusion was observed, especially between pawns and kings, as well as between bishops and queens, reinforcing the limitations in the generalization capability of the model trained exclusively with a small number of real data samples.



(a) Loss function during training



(b) Accuracy during training

FIGURE 18. Metrics evaluated during training of the model with only real images.

These comparative results highlight the importance of pre-training with synthetic images. This initial stage allows the model to learn fundamental characteristics in a robust and generalizable manner before being refined with real images. Using synthetic data creates a controlled environment for initial learning, facilitating subsequent adaptation to the real domain and significantly reducing the need for extensive labor in capturing numerous images of different board positions to achieve similar results.

B. RESULTS OF MOVE PREDICTION

This section presents the results obtained with the developed model for predicting chess moves. The analysis is structured in two parts: first, the performance of the individual classifiers is evaluated in terms of accuracy and generalization; then, the combined pipeline is assessed on an independent test set across different game phases, examining how data structuring and curation influence final performance.

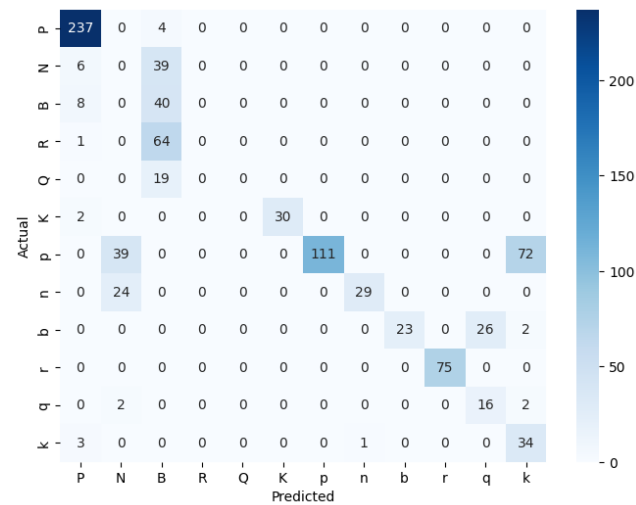


FIGURE 19. Confusion matrix of the classifier trained exclusively on real images.

1) RESULTS OF CLASSIFIER TRAINING

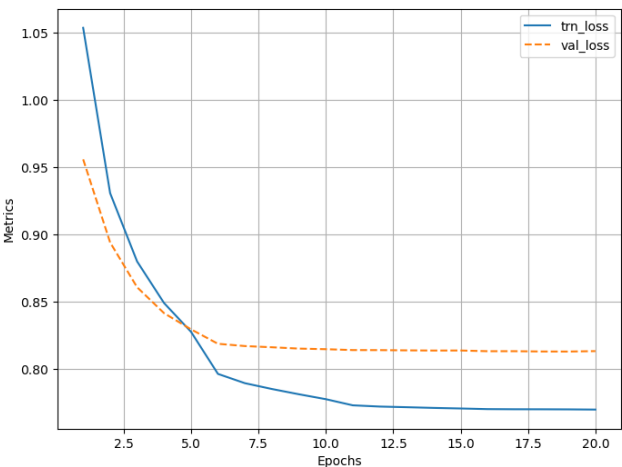
The individual classifiers were trained on curated, style-specific datasets and compared against the convolutional neural network approach of Oshri et al. [23]. The main methodological difference lies in the expansion and homogenization of the training data, which was tailored to reflect a specific playing style rather than general chess patterns. As shown in Table 9, this refinement resulted in training-to-validation accuracy gaps of only 1% to 3%, indicating effective generalization, in contrast with prior general-purpose studies, which faced larger discrepancies due to the high variability in chess playing styles.

TABLE 9. Accuracy for each instantiated model.

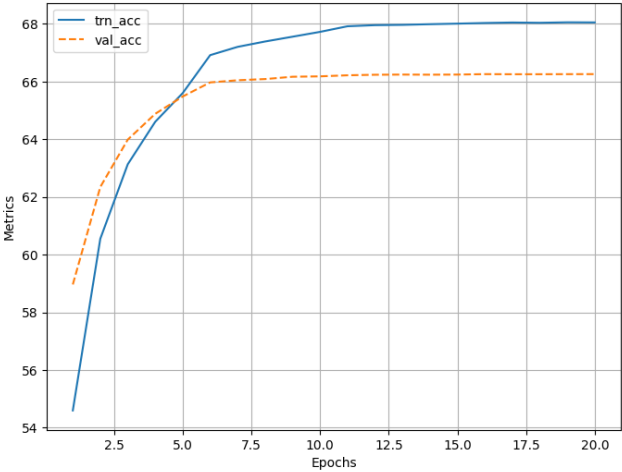
Prediction Model	Training (%)	Validation (%)
Piece to Move	68.05	66.26
Pawn Destination	80.57	78.42
Knight Destination	77.30	74.89
Bishop Destination	72.09	69.38
Rook Destination	50.51	49.22
Queen Destination	45.33	43.84
King Destination	72.65	70.11

The model for classifying the piece to be moved (Figure 20) showed promising results, with accuracies of 68.05% in the training set and 66.26% in the validation set. Although it handles only six classes, the task presents unique challenges, as the model must learn complex patterns that depend on the overall board position. The loss curves suggest good convergence throughout training, and the small gap between training and validation reinforces the model’s ability to handle different game positions.

When analyzing the destination classifiers for each piece, we observe a correlation between performance and the nature of each piece’s movements. Pieces with more restricted



(a) Loss function during training



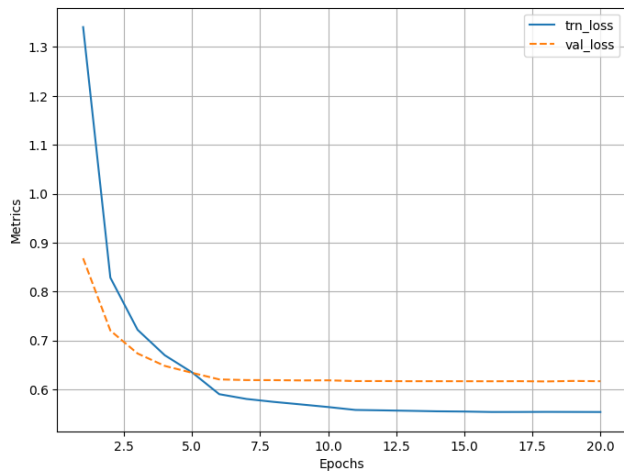
(b) Accuracy during training

FIGURE 20. Metrics evaluated during training of the piece-to-move classifier.

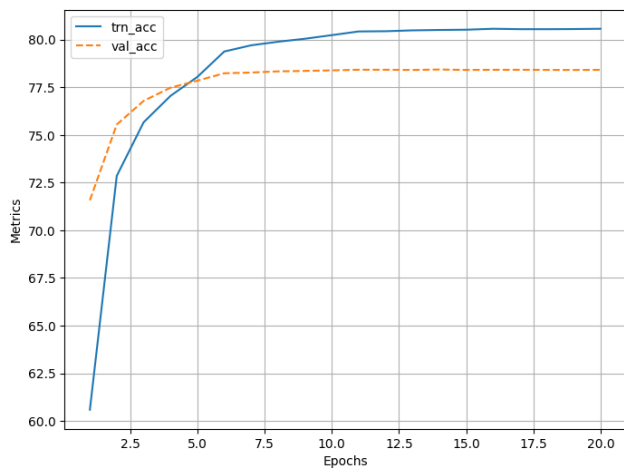
movement, such as the pawn (Figure 21), knight (Figure 22), and king (Figure 26), showed higher accuracies on both training and validation sets: 80.57% and 78.42% for the pawn, 77.30% and 74.89% for the knight, and 72.65% and 70.11% for the king. This occurs because these moves are more predictable and exhibit less variability.

On the other hand, pieces such as the queen (Figure 25) and rook (Figure 24) performed worse, with validation accuracies of 43.84% and 49.22%, respectively. These results reflect the greater freedom of movement of these pieces, increasing the complexity for the model to identify consistent patterns. The bishop (Figure 23), despite its relatively flexible movement, achieved better accuracies (69.38% on validation), likely due to its more frequent use early in the game, when it is more constrained by other pieces.

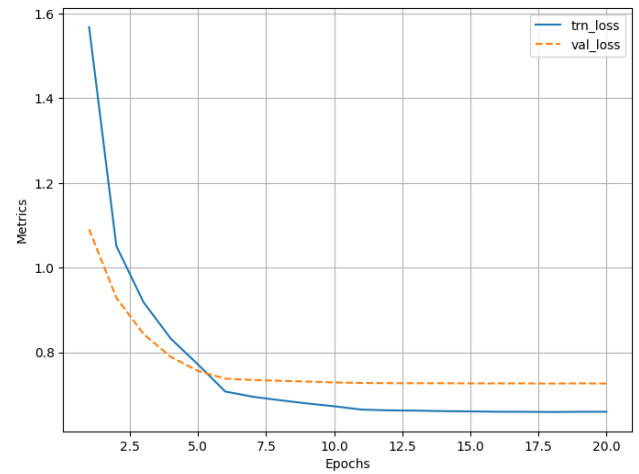
The results indicate that creating datasets targeted at specific playing styles is an effective approach to improve predictive model performance in chess. The reduction in the gap between training and validation suggests that the models



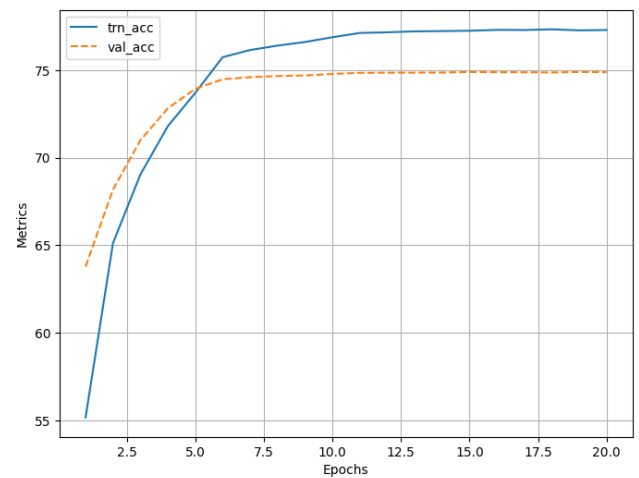
(a) Loss function during training



(b) Accuracy during training

FIGURE 21. Metrics evaluated during training of the pawn destination classifier.

(a) Loss function during training



(b) Accuracy during training

FIGURE 22. Metrics evaluated during training of the knight destination classifier.

are better adapted to the problem. Although some pieces pose greater difficulty due to the complexity of their movement patterns, the careful selection of examples helped mitigate some of this variability, leading to higher accuracy metrics than those observed in previous studies.

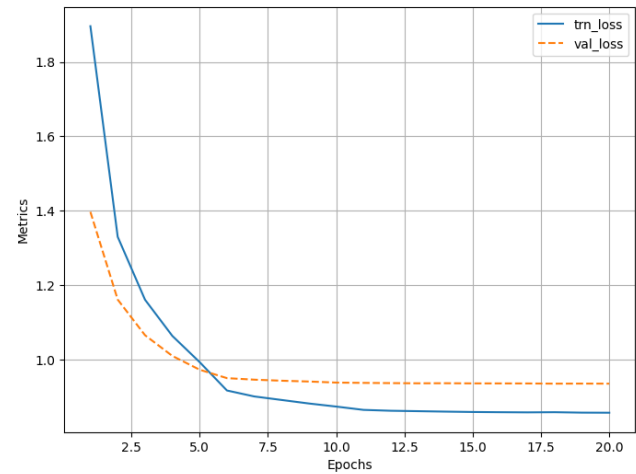
Furthermore, it is important to note that, although the number of classes for destination classifiers is fixed at 64 (all board squares), choosing the piece to move acts as a pre-filter, automatically restricting the relevant set of classes for each destination model. This interdependence contributed to more efficient performance, as evidenced by the good results obtained for pieces with more frequent or predictable moves.

2) RESULTS OF TESTING THE COMBINED MODELS

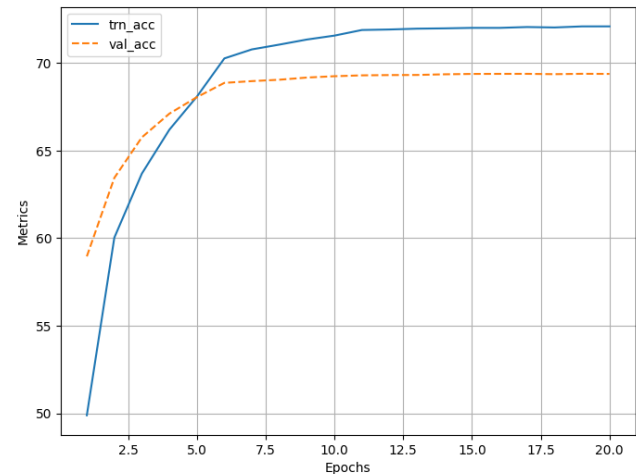
The full testing of the *pipeline* described in Section VI-D3a was finally carried out, evaluating its ability to predict human moves on an independent test set. The obtained results highlight the system's performance in comparison to previous approaches, such as Maia Chess [18]. The latter, based on

neural networks trained to predict human moves, achieved an accuracy of approximately 50%, while pure Stockfish reached only around 30% on the task of predicting plausible human moves.

In contrast, the method proposed in this research, which combines neural networks to filter moves with Stockfish as a validation mechanism, achieved superior results, reaching an average accuracy of 68.52%, as shown in Table 10. When compared directly with Stockfish under the same evaluation protocol, the proposed method consistently outperforms the engine across all game stages, with an average improvement of approximately 29 percentage points. It is important to note that this average was obtained without weighting, as each stage consisted equally of 2,000 test cases. This performance reflects not only the efficient integration between neural networks and the chess engine but also the strategy of using homogeneous datasets, allowing for more specialized and consistent training in specific game scenarios. In particular, the results indicate that the neural network component



(a) Loss function during training



(b) Accuracy during training

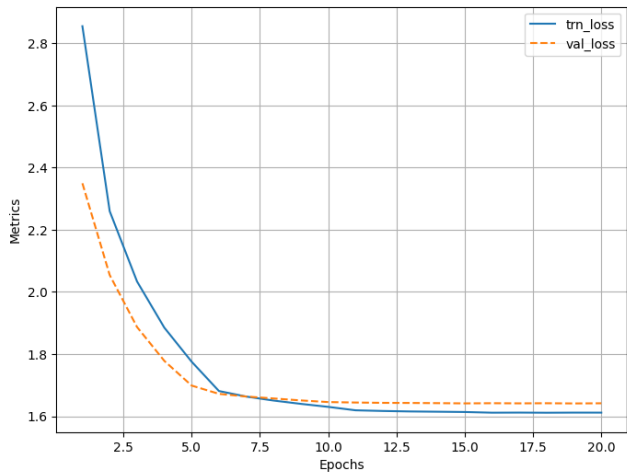
FIGURE 23. Metrics evaluated during training of the bishop destination classifier.

TABLE 10. Comparing the test accuracy (%) of the implemented model with Stockfish.

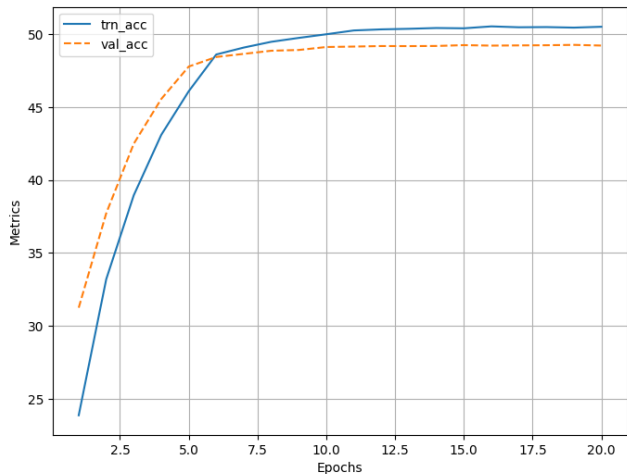
Game Stage	Implemented Model	Stockfish
Opening	99.65	67.80
Midgame	63.47	24.42
Endgame	42.43	25.93
Average	68.52	39.38

successfully biases the move selection toward human-like patterns, while Stockfish refines these candidates to ensure tactical consistency. This approach contrasts with previous studies that sought more generalist models, highlighting the potential of models focused on particular stages or contexts of the game.

A more detailed analysis of the stages reveals significant variations in performance. During the opening, the system achieved an extremely high accuracy of 99.65%. This represents a substantial improvement over Stockfish, which achieves 67.80% in the same setting, reinforcing



(a) Loss function during training



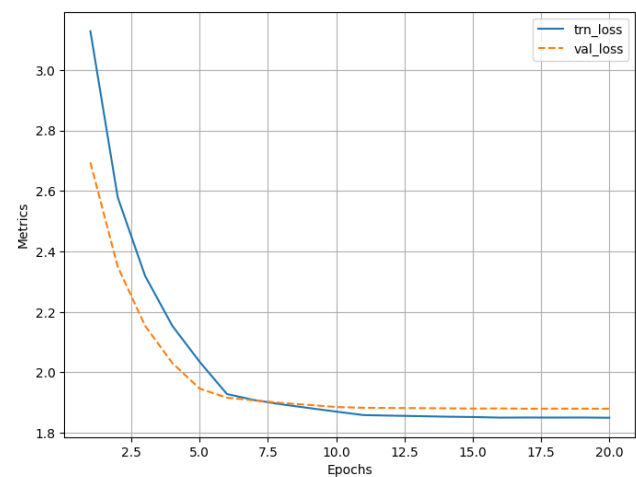
(b) Accuracy during training

FIGURE 24. Metrics evaluated during training of the rook destination classifier.

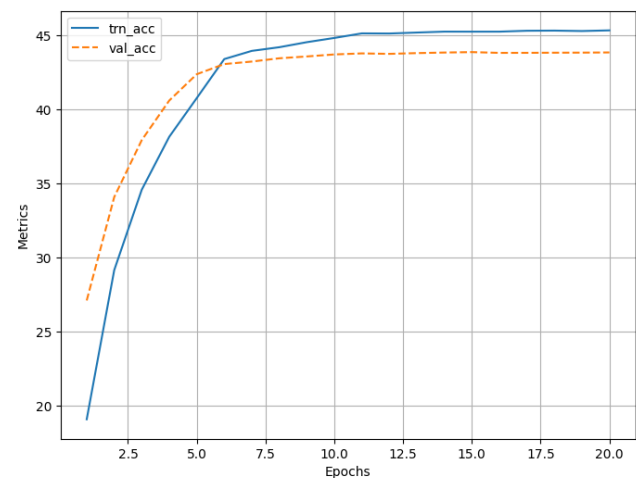
the effectiveness of the proposed approach in capturing human opening patterns. This result directly reflects the data homogenization strategy, where the selection of games with specific openings was prioritized to create a more standardized style of play and facilitate model training for this early game phase. The inherent predictability of openings, largely guided by well-established theories and patterns, also contributes to this superior performance.

In the midgame, accuracy dropped to 63.47%, reflecting the increasing tactical complexity and greater variability in human moves. Even in this more complex phase, the proposed method significantly outperforms Stockfish (24.42%), indicating that the learned human-like priors remain beneficial beyond highly structured positions. This decrease is expected, as the higher number of movable pieces and strategic possibilities make the scenario less standardized and more difficult to model.

In the endgame, accuracy was 42.43%, the lowest among all stages. Although the margin over Stockfish (25.93%) is smaller compared to earlier stages, the proposed method still



(a) Loss function during training



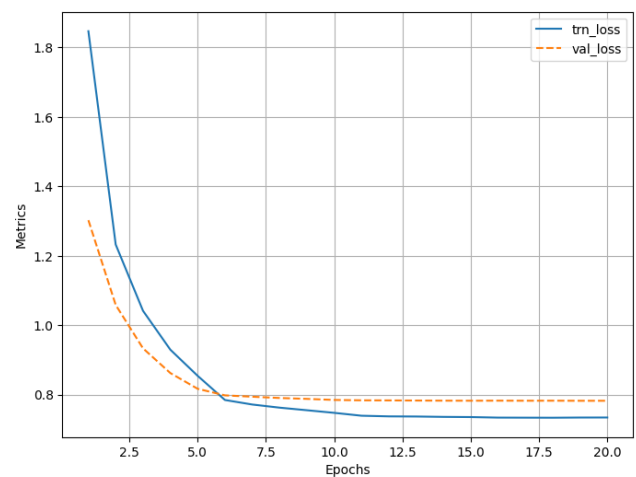
(b) Accuracy during training

FIGURE 25. Metrics evaluated during training of the queen destination classifier.

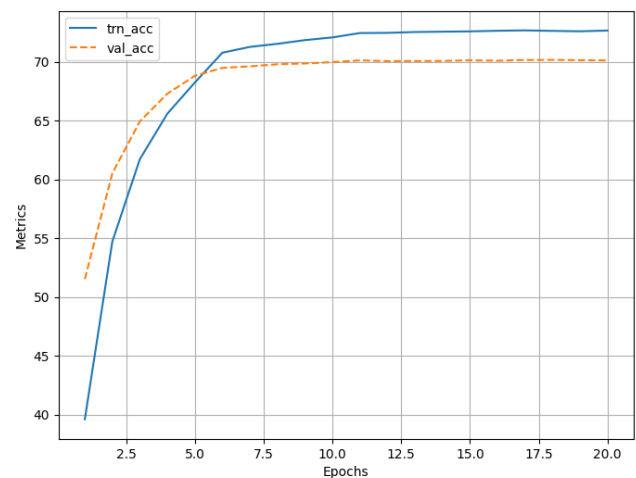
maintains a consistent advantage. This drop is associated with the higher unpredictability of moves in this phase, often influenced by emotional, psychological, or individual strategic decisions. Additionally, the smaller number of pieces and the unique scenarios that arise on the board make predicting moves even more challenging for any system.

These results reinforce the effectiveness of the *pipeline* in more structured stages of the game, such as the opening, and highlight the challenges in stages with greater variability, such as the endgame. The analysis suggests that specializing models by game stage could bring further improvements, pointing to the feasibility of training dedicated models for different phases, which could further increase accuracy and applicability in complex scenarios.

To further evaluate the generalization capability of the proposed approach, additional experiments were conducted on different openings not seen during training. The results obtained with the implemented model and with Stockfish are presented in Tables 11 and 12, respectively.



(a) Loss function during training



(b) Accuracy during training

FIGURE 26. Metrics evaluated during training of the king destination classifier.

TABLE 11. Test accuracy (%) for each game stage across different openings with the implemented model.

Game Stage	Ruy Lopez	Sicilian Defense	Queen’s Gambit
Opening	81.46	79.86	54.37
Midgame	53.62	39.93	37.98
Endgame	39.23	39.13	37.63
Average	58.10	52.97	43.33

Table 11 shows that, although the model was trained on a homogenized dataset, it is still able to maintain competitive performance across different openings. However, a decrease in accuracy is observed, particularly in the opening phase, reflecting the model’s specialization in the training distribution.

When comparing both tables, it becomes evident that the proposed method consistently outperforms Stockfish across all evaluated openings and game stages. For example, in the Ruy Lopez opening, the proposed model achieves an average

TABLE 12. Test accuracy (%) for each game stage across different openings with Stockfish.

Game Stage	Ruy Lopez	Sicilian Defense	Queen's Gambit
Opening	69.35	61.01	58.92
Midgame	26.13	27.48	21.13
Endgame	24.88	25.28	23.18
Average	40.12	37.92	34.41

accuracy of 58.10%, compared to 40.12% for Stockfish. Similar improvements are observed for the Sicilian Defense (52.97% vs. 37.92%) and Queen's Gambit (43.33% vs. 34.41%).

A more detailed analysis indicates that the largest performance gains occur during the midgame phase, where human decision-making is more diverse and less driven by established theory. In contrast, in the opening phase, especially in openings with strong theoretical foundations, the gap between the proposed method and Stockfish tends to decrease.

Overall, these results highlight a trade-off between specialization and generalization. While the model benefits significantly from training on a homogenized dataset, leading to high performance in the target domain, it still retains the ability to generalize to different openings while maintaining a consistent advantage over Stockfish. This reinforces the effectiveness of the hybrid approach, combining human-like priors with engine-based validation.

3) ABLATION STUDY AND CANDIDATE-MOVE RANKING

To assess the contribution of each component in the move-prediction pipeline, we performed an ablation study comparing three configurations: the CNN-only model, Stockfish-only, and the proposed hybrid approach. In addition, because the system naturally produces a ranked list of candidate moves, we report Top-1, Top-2, and Top-3 accuracies. For the CNN-only and hybrid configurations, the results are averaged over 5 runs with different random seeds and reported as mean $\pm$ standard deviation.

TABLE 13. Average Top- k accuracy (%) across game stages.

Model	Top-1	Top-2	Top-3
CNN	64.51 $\pm$ 0.40	72.92 $\pm$ 0.44	77.84 $\pm$ 0.51
Stockfish	39.38	46.84	60.87
Hybrid	68.49 $\pm$ 0.44	73.55 $\pm$ 0.44	78.64 $\pm$ 0.51

Table 13 shows that the proposed hybrid method achieves the best Top-1 result on average, improving over the CNN-only baseline by about 4 percentage points and over Stockfish by a much larger margin. This indicates that the integration with Stockfish is especially useful when the evaluation is restricted to the most likely move, i.e., the move that is actually selected as the final recommendation. However, the advantage of the hybrid model becomes much

smaller at Top-2 and Top-3, where the CNN-only model is already very competitive. This suggests that the neural network is the main source of candidate-move ranking quality, while Stockfish mainly helps refine the final choice among the most probable alternatives.

The smaller gain at Top-2 and Top-3 also indicates that the candidate list already contains a strong set of plausible human-like moves before Stockfish validation. In other words, the engine contributes more as a final filter than as the primary source of predictive power. This behavior is consistent with the intended design of the pipeline, in which the neural model generates a ranked human-like move distribution and Stockfish acts as a tactical validator.

TABLE 14. Top-1 accuracy (%) by game stage.

Model	Opening	Midgame	Endgame
CNN	99.68 $\pm$ 0.03	61.50 $\pm$ 0.46	32.33 $\pm$ 0.89
Stockfish	67.80	24.42	25.93
Hybrid	99.68 $\pm$ 0.03	63.57 $\pm$ 0.57	42.23 $\pm$ 0.78

Table 14 highlights the strongest effect of Stockfish in Top-1 accuracy. The opening phase is already highly saturated for the CNN-only model, which reaches 99.68% and remains unchanged in the hybrid setting. The most meaningful gains appear in the midgame and, especially, in the endgame, where the hybrid model improves substantially over the CNN-only baseline. This behavior suggests that Stockfish is most useful in positions where the neural model is more uncertain and where tactical validation can correct highly suboptimal choices.

At the same time, the CNN-only model already outperforms Stockfish in every stage, which confirms that the learned representation captures human-like move patterns better than the engine alone under this evaluation protocol. The hybrid model does not replace this behavior; rather, it sharpens it, mainly by preventing occasional poor selections in more complex positions.

TABLE 15. Top-2 accuracy (%) by game stage.

Model	Opening	Midgame	Endgame
CNN	99.87 $\pm$ 0.03	72.79 $\pm$ 0.85	46.11 $\pm$ 0.56
Stockfish	71.32	34.17	35.02
Hybrid	99.87 $\pm$ 0.03	73.54 $\pm$ 0.45	47.25 $\pm$ 1.01

For Top-2, the differences between CNN-only and hybrid become very small. This indicates that once the correct move is allowed to appear within the first two candidates, the neural model already captures most of the useful ranking structure. Stockfish does not substantially alter this ordering, which suggests that its main contribution is not to reorder the entire candidate set, but to refine the most critical decision at the top of the list.

Stockfish alone remains clearly below both neural configurations, particularly in the midgame, where the gap is large.

This reinforces the idea that a purely engine-based ranking is not sufficient to model human move preferences under the proposed evaluation protocol.

TABLE 16. Top-3 accuracy (%) by game stage.

Model	Opening	Midgame	Endgame
CNN	99.88 ± 0.03	78.01 ± 0.35	55.62 ± 1.28
Stockfish	77.55	51.61	53.46
Hybrid	99.90 ± 0.00	80.03 ± 0.55	56.02 ± 1.03

Top-3 results further confirm that the CNN-based ranking already covers most of the plausible human move space. The hybrid model remains marginally better in the opening and midgame, but the differences are small, and in the endgame the CNN-only model is slightly higher. This pattern indicates that Stockfish is not the dominant factor in candidate coverage; instead, it acts as a local correction mechanism that mostly improves the top decision while leaving the broader ranking largely unchanged.

Overall, the ablation study shows that the proposed system is robust across five random seeds and that its main advantage lies in improving the final Top-1 prediction, especially in the more difficult phases of the game. The CNN component contributes most of the predictive structure, while Stockfish improves tactical consistency and helps suppress rare but severe errors. This is precisely the intended behavior of the pipeline: generate a human-like ranked list of candidate moves, then use Stockfish to validate and refine the final recommendation.

C. ANALYSIS OF THE COMPLETE PIPELINE

This section presents the full application of the developed pipeline, from the input of a real board image (Figure 27) to move suggestions. The image was digitized, resulting in the FEN representation shown in Figure 28. This process demonstrates the system’s ability to accurately recognize the board configuration and convert it into a format usable by analysis models.

With the digitized position, move suggestions were generated. The developed pipeline suggested the move



FIGURE 27. Real board image used for move suggestion.

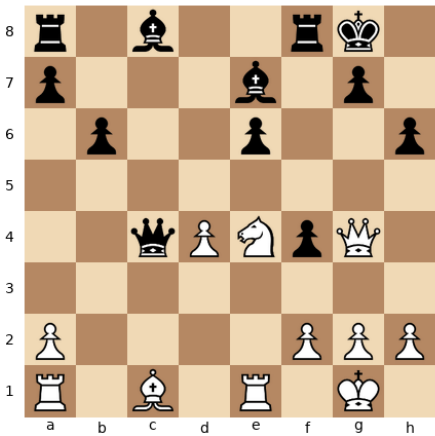
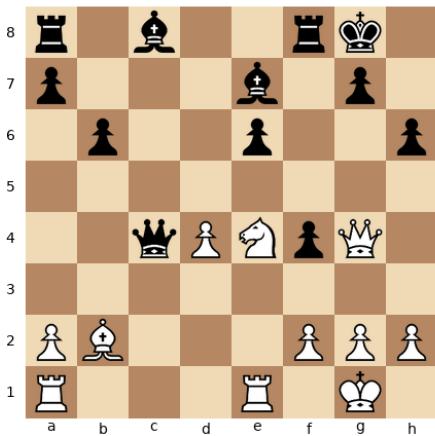
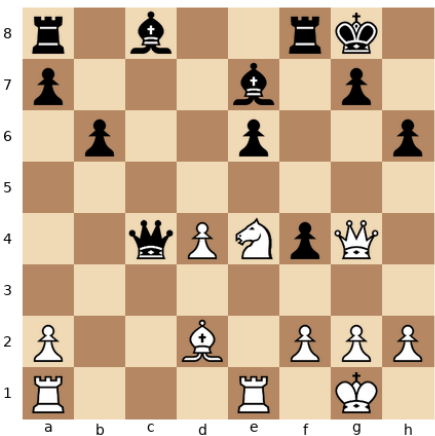


FIGURE 28. Board reconstructed from FEN: r1b2rk1/p3b1p1/1p2p2p/8/2qPNpQ1/8/P4PPP/R1B1R1K1.

“c1b2” (Figure 29(a)), while the best move identified by Stockfish was “c1d2” (Figure 29(b)). These results allow a comparative analysis between the approaches, highlighting differences in logic and strategy.



(a) Move suggested by the developed pipeline.



(b) Best move from Stockfish.

FIGURE 29. Positions resulting from possible moves.

The move “c1d2”, suggested by Stockfish, reflects a long-term tactical decision involving the potential sacrifice of the pawn on “d4”. Although this choice may not appear intuitive at first glance, it is supported by the engine’s high level of positional calculation, anticipating potential strategic advantages arising from this move. On the other hand, the move “c1b2” proposed by the developed *pipeline* follows a simpler and more accessible approach, protecting the pawn on “d4” while creating a direct checkmate threat if the opponent fails to respond adequately. This scenario is illustrated in Figure 30, where a careless sequence by the opponent could lead to checkmate. Although multiple defenses are possible and identified by the engine, the move suggested by the *pipeline* represents a solid and intuitive choice for a human player, requiring only a basic and direct understanding of the position.

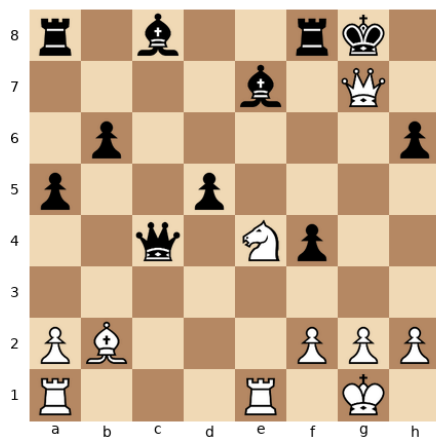


FIGURE 30. Possible final outcome.

The results of this analysis highlight two key aspects of the *pipeline*: the ability to suggest moves that balance human-like predictability with strategic feasibility, and the limitations imposed by the absence of deep calculation, as performed by Stockfish. The move suggested by the developed *pipeline*, while less optimized from a computational standpoint, aligns with more human-like play patterns, emphasizing a balance between safety and immediate attack.

VIII. CONCLUSION AND FUTURE WORK

This research work presented promising results in two main areas: the digitization of chess boards and the prediction of human moves. Based on the experiments conducted, the research questions described in Section IV can be addressed.

Regarding board digitization, the use of realistic 3D models generated through photogrammetry proved highly effective for creating image datasets for training. This approach demonstrated that synthetic models can significantly reduce the need for large sets of real images without compromising system performance. However, it is important to note that the dataset includes a significant portion of synthetic images, which may not fully capture real-world variability such as lighting conditions, occlusions, and variations in chess

piece design. Although the performance achieved with synthetic models did not directly translate to real images, refinement with real photographs was necessary on a smaller scale, validating the feasibility of the technique to reduce costs and data collection efforts. This indicates that the generalization of the digitization model to unseen real-world scenarios may still be limited. Furthermore, the developed 3D model set constitutes an important contribution for future experiments under controlled and diverse conditions. Additionally, methodologies could be integrated for detecting the board in any position within an image, using perspective projections to extract and centralize the region of interest. Such an approach would enhance the robustness of the solution, making it less susceptible to framing variations.

Concerning human move prediction, the analysis highlighted the positive impact of filtering training data based on player style or skill level. This strategy resulted in more homogeneous datasets, facilitating model learning, but introduced a bias towards a specific playing style. In particular, the dataset consists of games following specific openings, the Italian Opening, which bias the model toward recurring positional patterns and typical move sequences associated with these lines. Additionally, the dataset exhibits class imbalance, with a higher frequency of pawns compared to other pieces, which may still introduce residual bias despite the use of mitigation strategies such as weighted sampling. Nonetheless, the complexity and wide diversity of chess remain significant challenges. Another limitation is the factorized representation of moves, where origin and destination squares are modeled independently, which may restrict the ability of the model to capture dependencies between these variables. Furthermore, the combination of piece and destination probabilities using a simple arithmetic mean, although effective in practice, is a heuristic approach that may not fully capture the joint distribution of valid moves. An additional limitation arises when multiple identical pieces can legally move to the same destination square (e.g., two rooks moving to the same square), as these moves receive identical probabilities in the current formulation, preventing the model from distinguishing between them without external evaluation. For future work, it is suggested to group games by style or game phase, which may improve performance and allow the creation of specialized models for specific contexts. Future investigations should explore more principled alternatives for combining these probabilities, as well as mechanisms to explicitly model origin squares or disambiguate identical pieces, such as product-based formulations with normalization or learned fusion mechanisms, which may better capture the dependencies between piece selection and destination prediction. To further expand the analysis of the results, it would be interesting to validate the insights with players of different skill levels.

Finally, the integration of the model with the Stockfish engine demonstrated the potential of a hybrid approach. This combination aligned human intuition, captured by the model, with Stockfish’s strategic precision, resulting in predictions

that balance realistic playability and tactical effectiveness. However, this approach also introduces a dependency on engine-based evaluation, which may constrain the diversity of moves and limit the exploration of alternative human-like strategies. This methodology opens the way for the development of advanced chess analysis and decision-support tools, innovatively exploring the synergy between artificial intelligence and chess engines.

Beyond the chess domain, the proposed hybrid paradigm highlights a broader contribution to artificial intelligence: the integration of data-driven models with rule-based or optimization-based systems to balance interpretability and performance. This type of architecture can be extended to other application domains where purely optimal solutions are not necessarily aligned with human reasoning or usability. For instance, in educational technologies, such systems could recommend solutions or learning paths that are not only correct but also pedagogically appropriate for the learner's level. In the specific context of chess education, this approach enables the development of training tools that suggest moves aligned with human thinking, helping players understand strategic concepts rather than merely following engine-optimal lines.

In the field of entertainment, this methodology also enables the creation of more realistic and engaging AI opponents capable of simulating the playing styles of human players, including professional-level behavior. Such systems could be used in digital platforms and games to reproduce distinct strategic profiles, providing more immersive experiences and enabling users to practice against opponents that mimic real-world play styles rather than purely optimal engines.

In healthcare decision-support systems, hybrid approaches may help reconcile statistically optimal predictions with clinically interpretable recommendations, improving trust and adoption. Similarly, in human-in-the-loop systems, such as assistive robotics or interactive planning tools, combining learned human-like priors with formal validation mechanisms can enhance collaboration between humans and AI systems.

In addition, the digitization pipeline based on synthetic data generation and photogrammetry can be generalized to other computer vision tasks where annotated real-world data is scarce or expensive to obtain. Applications may include industrial inspection, cultural heritage preservation, and augmented reality systems, where controlled synthetic environments can significantly accelerate model development while reducing data acquisition costs.

Future work should therefore not only investigate more diverse datasets, alternative move representations, and more comprehensive evaluation protocols, but also explore the adaptation of this hybrid framework to other domains, assessing its impact on generalization, interpretability, and human-AI interaction across different application scenarios.

REFERENCES

- [1] S. Russel and P. Norvig, *Artificial Intelligence*. Upper Saddle River, NJ, USA: Prentice-Hall, 2004, p. 16.
- [2] C. E. Shannon, "XXII. Programming a computer for playing chess," *London, Edinburgh, Dublin Phil. Mag. J. Sci.*, vol. 41, no. 314, pp. 256–275, Mar. 1950.
- [3] S. H. Fuller, J. G. Gaschnig, and J. Gillogly, "Analysis of the alpha-beta pruning algorithm," Dept. Comput. Sci., Carnegie-Mellon University, Pittsburgh, PA, USA, 1973.
- [4] A. Turing, "Intelligent machinery (1948)," New York, NY, USA: Oxford University Press, pp. 395–432, 2004.
- [5] M. Newborn, *Kasparov Versus Deep Blue: Computer Chess Comes of Age*. Cham, Switzerland: Springer, 2012.
- [6] P. W. Frey, "Second appendix CHESS 4.5 and CHESS 4.6: Competition in 1977 and 1978," in *Chess Skill in Man and Machine*. Cham, Switzerland: Springer, 1983, pp. 248–256.
- [7] F.-H. Hsu, *Behind Deep Blue: Building the Computer That Defeated the World Chess Champion*. Princeton, NJ, USA: Princeton Univ. Press, 2002.
- [8] G. Kendall and G. Whitwell, "An evolutionary approach for the tuning of a chess evaluation function using population dynamics," *Congr. Evol. Comput. (IEEE Cat. No. 01TH8546)*, vol. 2, pp. 995–1002, May 2001.
- [9] M. Lai, "Giraffe: Using deep reinforcement learning to play chess," 2015, *arXiv:1509.01549*.
- [10] C. B. Browne, E. Powley, D. Whitehouse, S. M. Lucas, P. I. Cowling, P. Rohlfshagen, S. Tavener, D. Perez, S. Samothrakakis, and S. Colton, "A survey of Monte Carlo tree search methods," *IEEE Trans. Comput. Intell. AI Games*, vol. 4, no. 1, pp. 1–43, Mar. 2012.
- [11] M. C. Fu, "AlphaGo and Monte Carlo tree search: The simulation optimization perspective," in *Proc. Winter Simul. Conf. (WSC)*, Dec. 2016, pp. 659–670.
- [12] O. E. David, N. S. Netanyahu, and L. Wolf, "DeepChess: End-to-end deep neural network for automatic learning in chess," in *Proc. Int. Conf. Artif. Neural Netw.*, 2016, pp. 88–96.
- [13] G. Hinton, "Deep belief networks," *Scholarpedia*, vol. 4, no. 5, p. 5947, 2009.
- [14] S. Benhur. (2020). *A Friendly Introduction to Siamese Networks*. [Online]. Available: <https://towardsdatascience.com/a-friendly-introduction-to-siamese-networks-85ab17522942>
- [15] D. Silver, T. Hubert, J. Schrittwieser, I. Antonoglou, M. Lai, A. Guez, M. Lanctot, L. Sifre, D. Kumaran, T. Graepel, T. Lillicrap, K. Simonyan, and D. Hassabis, "A general reinforcement learning algorithm that masters chess, shogi, and go through self-play," *Science*, vol. 362, no. 6419, pp. 1140–1144, Dec. 2018.
- [16] Stockfish Contributors. (2023). *Stockfish Chess Engine—Versão 16*. [Online]. Available: <https://stockfishchess.org>
- [17] A. Champion. (2021). *Dissecting Stockfish Part 2: In-Depth Look At a Chess Engine*. [Online]. Available: <https://medium.com/data-science/dissecting-stockfish-part-2-in-depth-look-at-a-chess-engine-2643cdc35c9a>
- [18] R. McIlroy-Young, S. Sen, J. Kleinberg, and A. Anderson, "Aligning superhuman AI with human behavior: Chess as a model system," in *Proc. 26th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Aug. 2020, pp. 1677–1687.
- [19] H. J. Berliner, "Chess as problem solving: The development of a tactics analyzer," Dept. Comput. Sci., Carnegie-Mellon University, Pittsburgh, PA, USA, Tech. Rep. AL-784 881, 1974.
- [20] G. Goetsch and M. S. Campbell, "Experiments with the null-move heuristic," in *Computers, Chess, and Cognition*. Cham, Switzerland: Springer, 1990, pp. 159–168.
- [21] G. P. Kjartansson, "Experimenting with time and depth control in chess engines," Dept. Comput. Sci., Reykjavik University, Iceland, 2019.
- [22] G. Tesauro, "Temporal difference learning and TD-gammon," *Commun. ACM*, vol. 38, no. 3, pp. 58–68, 1995.
- [23] B. Oshri and N. Khandwala, "Predicting moves in chess using convolutional neural networks," Dept. Comput. Sci., Stanford University, Course Project Reports CS231, 2015.
- [24] Z. Kevanishvili and M. Iavich, "A hybrid human-centric framework for discriminating engine-like from human-like chess play: A proof-of-concept study," *Appl. Syst. Innov.*, vol. 9, no. 1, p. 11, Dec. 2025.
- [25] B. J. Bulsink, "Device for detecting playing pieces on a board," Patent U.S. 6 168 158 B1, Jan. 2, 2001.
- [26] M. Breazu and C. F. Paèzanghel, "A robotic arm for playing chess," *Int. J. Adv. Statist. IT Econ. Life Sci.*, vol. 14, no. 1, pp. 79–90, Dec. 2024.
- [27] K. Matuszek, B. Mayton, R. Aimi, M. P. Deisenroth, L. Bo, R. Chu, M. Kung, L. LeGrand, J. R. Smith, and D. Fox, "Gambit: An autonomous chess-playing robotic system," in *Proc. IEEE Int. Conf. Robot. Autom.*, May 2011, pp. 4291–4297.

- [28] J. Ding, "ChessVision: Chess board and piece recognition," Dept. Comput. Sci., Stanford University, Course Project Reports CS231, 2016.
- [29] D. M. Quintana, A. A. D. B. García, and M. P. Matías, "LiveChess2FEN: A framework for classifying chess pieces based on CNNs," 2020, *arXiv:2012.06858*.
- [30] A. Masouris and J. van Gemert, "End-to-end chess recognition," 2023, *arXiv:2310.04086*.
- [31] L. Spears. (2019). *Transcribe Live Chess With Machine Learning—Part 1*. [Online]. Available: <https://medium.com/data-science/transcribe-live-chess-with-machine-learning-part-1-928f73306e1f>
- [32] P. Karia, V. Jain, M. Shah, and S. Rane, "Digitization of chess board and prediction of next move," in *Proc. IEEE 7th Int. Conf. Conver. Technol. (I2CT)*, Apr. 2022, pp. 1–5.
- [33] G. Wölflein and J. Arandjelović, "Determining chess game state from an image," *J. Imag.*, vol. 7, no. 6, p. 94, Jun. 2021.
- [34] H. Lakkaraju, S. H. Bach, and J. Leskovec, "Interpretable decision sets: A joint framework for description and prediction," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Aug. 2016, pp. 1675–1684.
- [35] Z. Tang, D. Jiao, R. McIlroy-Young, J. Kleinberg, S. Sen, and A. Anderson, "Maia-2: A unified model for human-AI alignment in chess," *Adv. Neural Inf. Process. Syst.*, vol. 37, pp. 20919–20944, Dec. 2024.
- [36] D. Zhong, D. Huang, and C. Greenberg, "Predicting human chess moves: An AI assisted analysis of chess games using skill-group specific n -gram language models," 2025, *arXiv:2512.01880*.
- [37] E. G. Hari Sekar and R. Jin, "Human-aligned chess AI: A multitask transformer for humanlike decision-making," in *Proc. IEEE Conf. Artif. Intell. (CAI)*, May 2025, pp. 1230–1234.
- [38] K. Miyake, "Evaluating the reliability of three-dimensional models constructed photogrammetry software 3DF zephyr by measuring joint angles of fingers: A comparison to a conventional goniometer," *J. Plastic Reconstructive Surgery*, vol. 3, no. 1, pp. 34–38, 2024.
- [39] 3Dflow. (2024). *3DF Zephyr Official Documentation*. [Online]. Available: <https://www.3dflow.net/technology/documents/>
- [40] D. G. Lowe, "Distinctive image features from scale-invariant keypoints," *Int. J. Comput. Vis.*, vol. 60, no. 2, pp. 91–110, Nov. 2004.
- [41] H. Bay, T. Tuytelaars, and L. Van Gool, "SURF: Speeded up robust reatures," in *9th Eur. Conf. Comput. Vis.*, pp. 404–417, Cham, Switzerland: Springer, May 2006.
- [42] M. A. Fischler and R. C. Bolles, "Random sample consensus: A paradigm for model fitting with applications to image analysis and automated Cartography," *Commun. ACM*, vol. 24, no. 6, pp. 381–395, 1981.
- [43] O. Özyeğil, V. Voroninski, R. Basri, and A. Singer, "A survey of structure from motion," *Acta Numerica*, vol. 26, pp. 305–364, May 2017.
- [44] J. L. Schönberger and J.-M. Frahm, "Structure-from-motion revisited," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 4104–4113.
- [45] Python Software Foundation. (2024). *Python Programming Language*. [Online]. Available: <https://www.python.org/>
- [46] Facebook AI Research. (2024). *PyTorch*. [Online]. Available: <https://pytorch.org/>
- [47] G. Bradski. (2000). *OpenCV*. [Online]. Available: <https://opencv.org/>
- [48] J. D. Hunter. (2007). *Matplotlib*. [Online]. Available: <https://matplotlib.org/>
- [49] 3Dflow SRL. (2024). *3DF Zephyr*. [Online]. Available: <https://www.3dflow.net/3df-zephyr-photogrammetry-software/>
- [50] Blender Foundation. (2024). *Blender*. [Online]. Available: <https://www.blender.org/>
- [51] N. Fiekas. (2024). *Python-Chess*. [Online]. Available: <https://python-chess.readthedocs.io/>
- [52] (2024). *Open Database Lichess Standard Chess*. [Online]. Available: <https://database.lichess.org/#standardgames>
- [53] D. J. Barnes and J. Hernandez-Castro, "On the limits of engine analysis for cheating detection in chess," *Comput. Secur.*, vol. 48, pp. 58–73, Feb. 2015.
- [54] M. Iavich and Z. Kevanishvili, "Detecting fair play violations in chess using neural networks," in *Proc. Int. Conf. Inf. Soc. Univ. Stud. (IVUS)*, 2024, pp. 121–127.
- [55] I. Cheng and C. Camargo, "Machine learning to study patterns in chess games," *J. Artif. Intell. Res.*, vol. 74, pp. 123–138, Jan. 2023.
- [56] J. Canny, "A computational approach to edge detection," *IEEE Trans. Pattern Anal. Mach. Intell.*, vols. PAMI–8, no. 6, pp. 679–698, Nov. 1986.
- [57] P. V. Hough, "Method and means for recognizing complex patterns," U.S. Patent 3 654 069, Dec. 18, 1962.
- [58] M. Abdulrazaq, "Rare-event prediction in imbalanced data: A unified evaluation and optimization framework for high-risk systems," *Commun. Phys. Sci.*, vol. 9, no. 4, pp. 968–979, 2023.
- [59] H. Panchal, S. Mishra, and V. Shrivastava, "Chess moves prediction using deep learning neural networks," in *Proc. Int. Conf. Adv. Comput. Commun. (ICACC)*, Oct. 2021, pp. 1–6.
- [60] S. Narayanan, K. Korley, C.-J. Ho, and S. Sen, "Improving the strength of human-like models in chess," *Int. Conf. Learn. Representations*, pp. 1–12, Feb. 2023.



HEIGON SOLDERA received the M.Sc. degree in computer science from the University of Campinas (Unicamp), Brazil, in 2025, where he is currently pursuing the Ph.D. degree. His research interests include machine learning, natural language processing, and computer vision.



HELIO PEDRINI (Senior Member, IEEE) received the B.Sc. degree in computer science and the M.Sc. degree in electrical engineering from the University of Campinas (Unicamp), Brazil, and the Ph.D. degree in electrical and computer engineering from the Rensselaer Polytechnic Institute (RPI), Troy, NY, USA. He is currently a Full Professor with the Institute of Computing, Unicamp. His research interests include machine learning, image processing, computer vision, and computer graphics.

...

Coordenação de Aperfeiçoamento de Pessoal de Nível Superior (CAPES) - ROR identifier: 00x0ma614